

Microsoft® .NET Explained

¿Por qué Microsoft® .NET?

Por Daniel Rubiolo, J.D. Meier, Edward Jezierski, Alex Mackman

Resumen

Este artículo describe las tecnologías que constituyen la plataforma .NET y las ventajas que ofrecen a los desarrolladores.

Este es un documento preliminar y puede sufrir cambios sustanciales antes de la publicación comercial definitiva del software que aquí se describe.

La información contenida en este documento representa la visión actual de Microsoft Corporation en los asuntos analizados a la fecha de publicación. Debido a que Microsoft debe responder a las cambiantes condiciones de mercado no deberá interpretarse como un compromiso por parte de Microsoft, y la compañía no puede garantizar la exactitud de la información presentada después de la publicación.

Este Documento estratégico es sólo para fines informativos. MICROSOFT NO OFRECE NINGUN TIPO DE GARANTIA, EXPRESA O IMPLICITA EN ESTE DOCUMENTO.

El cumplimiento de todas las leyes de derechos de autor aplicables es responsabilidad del usuario, sin limitar los derechos de autor, ninguna parte de este documento puede ser reproducida, almacenada o introducida en un sistema de recuperación, o transmitida en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, registro, o cualquier otra forma) o para cualquier fin, sin el consentimiento expreso escrito de Microsoft Corporation.

Microsoft puede tener patentes, aplicaciones de patentes, marcas registradas, derechos de autor u otros derechos de propiedad intelectual que cubren el tema que se trata en este documento. Excepto como se establezca expresamente en cualquier acuerdo de licencia escrito por parte de Microsoft, este documento no le otorga ningún tipo de licencia a estas patentes, marcas registradas, derechos de autor u otra propiedad intelectual.

© 2001 Microsoft Corporation. Todos los derechos reservados.

Microsoft, bCentral, Hotmail, MSN, Authenticode, IntelliSense, el logotipo .NET, Windows son registros o marcas registradas de Microsoft Corporation en los Estados Unidos y/o otros países.

Los nombres de compañías reales y productos mencionados en este documento pueden ser marcas registradas de sus respectivos propietarios.

00x01

TABLA DE CONTENIDOS

INTRODUCCION	1
ARQUITECTURA DEL ENTORNO .NET	2
Internet como infraestructura de aplicaciones	2
Principales Componentes .NET	2
Permitiendo servicios Web XML	3
Distribuyendo e integrado lógica de aplicación a través de Internet	4
Formatos de descripción de datos estándar para servicios Web	5
Utilizando estándares abiertos de la industria	6
Una arquitectura tecnológica actualizada y moderna	6
EL ENTORNO .NET	8
El CLR simplifica el desarrollo	8
El entorno es multilenguaje y ampliable	8
Interoperabilidad cruzada entre lenguajes	8
Diseño multi-plataforma	9
La BCL reemplaza las APIs	9
Código administrado y ensambles	9
Características importantes integradas en el Framework	9
MODERNA TECNOLOGIA DE COMPILACIÓN	13
El MSIL hace que su código sea independiente de la CPU y del sistema operativo	13
MSIL es administrado por CLR	13
Compilación justo a tiempo mejorada	13
La Pre-Compilación dispara el desempeño	14
Actualizando componentes dinámicamente	15
Los espacios de nombre proporcionan un paradigma de programación consistente	15
El .NET Framework es ampliable	16
EL COMMON TYPE SYSTEM	17
CLR implementa un sistema de tipos genéricos	17
Límites de visibilidad flexibles para tipos	17
Comportamiento consistente de los objetos	18
Herencia simple de clase, herencia múltiple de interfaces	18
Integración e interoperación multilenguaje	19
CLS como subconjunto de CTS	19
Todos los lenguajes son iguales	19
El código administrado tiene un amplio soporte en la industria	20
INTRODUCCION A LA SEGURIDAD E IMPLEMENTACION	21
Un sistema de seguridad granular y basado en evidencias	21
Implementación de lado a lado	21
La seguridad es integral al diseño de .NET.	21

.NET COMO EVOLUCIÓN DE COM	27
Infraestructura entre componentes invisible	27
Los Componentes .NET y componentes COM son iguales	27
.NET y COM+	27
Administración automática	28
Explotando las inversiones COM existentes	28
Interoperabilidad con código no administrado	28
Interoperabilidad de .NET con funciones DLL no administradas	29
Interoperabilidad de .NET con COM	29
Interoperabilidad de código no administrado con .NET	29
CARACTERISTICAS DE IMPLEMENTACION	31
Ensamblés para versionamiento e implementación	31
La Metadata posibilita la mayoría de las características de la plataforma	31
La Metadata proporciona la integración de múltiples lenguajes	32
La Metadata permite la interoperabilidad	32
Los compiladores generan metadata	32
Los Componentes de un ensamble son descriptos en Manifestos	33
Los ensambles determinan el alcance de los módulos	33
El caché de ensambles mejora el desempeño	33
BENEFICIOS PARA EL PROGRAMADOR	35
Programar Internet con servicios Web XML	35
Acceso a datos para servicios Web	35
Desarrollo avanzado de aplicaciones Web con ASP .NET	36
Código más limpio	38
Compatibilidad del explorador	39
Modelo Sin Estado	40
Web Forms	42
Controles del servidor	43
Modelo orientado a objetos	44
Controles de usuario	45
Controles para Conexión a Datos	46
Controles de Validación	48
Servicios Web XML	48
Memoria caché mejorada	50
Depuramiento	51
Rastreo	51
Implementación	51
Escalabilidad y disponibilidad	52
Modelo de Aplicaciones de Formularios Windows	53
Herencia Visual	53
Diseño de Formularios de Precisión	53
Bajo Costo Total de Propiedad	54
Visual Studio.NET	54
Lenguajes	55

Opción de lenguaje	56
Visual Basic.NET	57
C# (C Sharp)	59
Extensiones administradas para C++	62
Mejoras en Acceso a Datos	64
DataSets de ADO.NET	64
Compartir Datos con ADO.NET	65
Acceso Desconectado a Bases de Datos	68
APENDICE	69
Vínculos a recursos	69
Tecnología	69
Implementación y soporte de usuarios	69
Desarrolladores	69
Sitios generales para visitar	70

INTRODUCCION

Microsoft® .NET proporciona un entorno sólido para el desarrollo de aplicaciones y brinda ventajas técnicas importantes para los desarrolladores. Una pregunta que comúnmente se hacen los desarrolladores es: "¿Por qué Microsoft .NET?"

Este artículo ayuda a responder esta pregunta y proporciona antecedentes sobre las diferentes tecnologías que constituyen la plataforma .NET. Asimismo, analiza las ventajas técnicas al desarrollar soluciones .NET.

Para más explicaciones relacionadas, consulte los otros artículos de la serie "Microsoft .NET Explicado":

- **Cambio de paradigma a computación distribuida a través de Internet.** En este artículo se describe cómo nuevas tecnologías, nuevas herramientas y nuevos enfoques para el desarrollo de aplicaciones permiten el cambio a un modelo de computación distribuida basado en Internet. Asimismo, analiza las condiciones del mercado y los avances tecnológicos que han sentado las bases para .NET.
- **¿Qué es Microsoft .NET?** Este artículo examina lo que significa la visión Microsoft® .NET para profesionales de tecnología y de negocios y para el futuro de las aplicaciones de software. Describe la visión .NET y explica las ventajas competitivas que brindará a los negocios.

ARQUITECTURA DEL ENTORNO .NET

Internet como infraestructura de aplicaciones

.NET es la plataforma de Microsoft para servicios Web XML, la siguiente generación de software que conecta nuestro mundo de información, dispositivos y personas de una manera unificada y personalizada.

La Plataforma .NET permite la creación y uso como servicios de aplicaciones, procesos y sitios Web basados en XML, que compartan y combinen información y funcionalidad entre ellos por diseño, en cualquier plataforma o dispositivo inteligente, para proveer soluciones a la medida para empresas e individuos.

La Plataforma .NET incluye una familia de productos integral, construida en estándares de la industria y de Internet, que provee servicios Web XML para cada aspecto del desarrollo (herramientas), administración (servers), uso (bloques ensamblables y clientes inteligentes) y experiencias (ricas experiencias de usuario). .NET será parte de las aplicaciones, herramientas, y servers de Microsoft que usa actualmente – así como será parte de nuevos productos que extienden las capacidades de los servicios Web XML a todas sus necesidades de negocio.

Microsoft .NET le permitirá crear programas que trascienden los límites de los dispositivos y fortalecen la conectividad de Internet en sus aplicaciones. Además, es viable para todas sus aplicaciones, ya que no necesita pensar en dos infraestructuras separadas—una para aplicaciones Web y otra para aplicaciones internas o de escritorio.

Principales Componentes .NET

Desde un punto de vista tecnológico, .NET es la plataforma y las experiencias .NET construídas sobre la plataforma. La plataforma incluye:

- Herramientas – para construir aplicaciones y servicios Web XML (.NET Framework y Visual Studio.NET)
- Servers – sobre los que construir, proveer y desplegar esas aplicaciones y servicios (Windows 2000 Server and the .NET Enterprise Servers)
- Servicios – un conjunto central de servicios .NET ensamblables (servicios “HailStorm”)
- Software cliente – el software que provee dispositivos inteligentes, permitiendo a los usuarios interactuar y experimentar la plataforma .NET
- Experiencias – la combinación de los componentes de la plataforma .NET nombrados permiten experiencias de usuario más personales e integradas – experiencias .NET.

Estos componentes principales de .NET están ilustrados en la Figura 1. Para una descripción más detallada de cada uno de ellos vea la sección “La Plataforma .NET” del artículo “¿Qué es Microsoft .NET?” de esta serie “Microsoft .NET Explicado”.

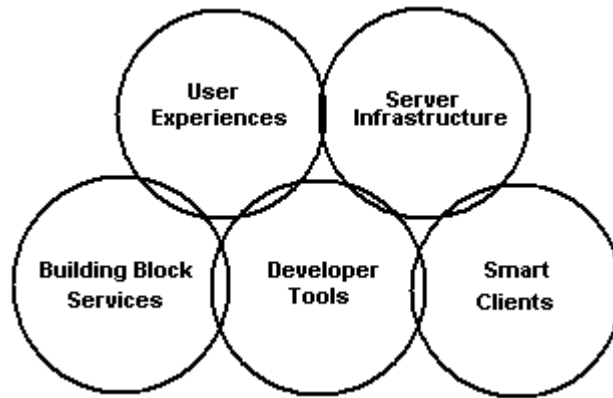


Figura 1: Componentes principales de .NET

Permitiendo servicios Web XML

Hoy en día, un número cada vez mayor de soluciones de software se ha desarrollado incluyendo sistemas complejos de n-niveles que con frecuencia integran varias aplicaciones heterogéneas que se extienden a través de WANs.

En el contexto de la computación capacitada por Internet, el software no es algo que se instala de un CD, sino que es un servicio (como el Caller ID o televisión donde paga por evento) para suscribirse a través de un medio de comunicaciones. Este nuevo tipo de software une los aspectos firmemente acoplados y altamente productivos de la computación de n-niveles con los conceptos del Web poco acoplados, orientados a mensajes, y utiliza el estándar XML de Internet como el formato en común para datos.

Los servicios Web XML materializan este estilo de computación. Un servicio Web XML es un componente de software o una aplicación que expone sus funciones programáticamente a través de Internet o la intranet utilizando los protocolos estándar de Internet como son Simple Object Access Protocol (SOAP) para comunicación entre programas, y XML para representación de datos. Puede ser de ayuda pensarlo como programación componentizada a través del Web. El modelo de servicios Web se muestra en la Figura 2.

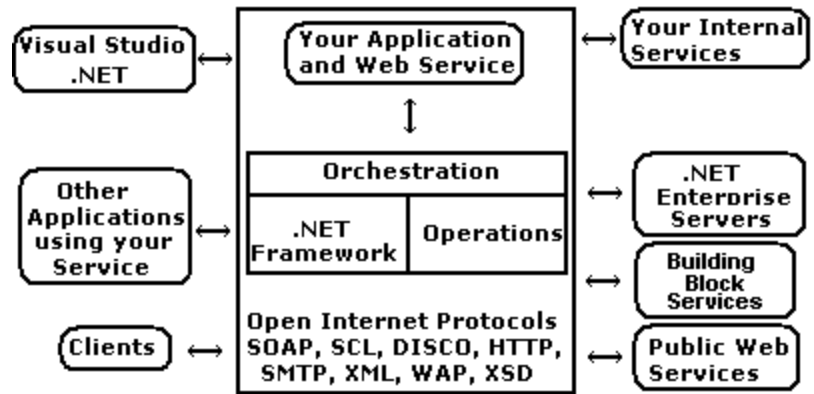


Figura 2: Modelo de Servicios Web

Conceptualmente, usted integra los servicios Web externos en sus aplicaciones enrutando las solicitudes a través de Internet a un servicio que reside en un sistema remoto. Por ejemplo, un servicio como Microsoft Passport podría permitirle proporcionar autenticación para una aplicación. Mediante la programación del servicio Passport, puede aprovechar la infraestructura de Passport y confiar en ella para mantener la base de datos de los usuarios asegurando que sea operada de manera eficaz, respaldada regularmente, etc.

Distribuyendo e integrado lógica de aplicación a través de Internet
El concepto de distribuir lógica aplicación a través de la red no es nuevo. Lo que es nuevo, es el concepto de distribuir e integrar lógica de aplicación a través de Internet.

Anteriormente, la lógica de aplicación distribuida requirió modelos de objetos distribuidos tales como Distributed Component Object Model (DCOM) de Microsoft, Common Object Request Broker Architecture (CORBA) del Object Management Group, o Remote Method Invocation (RMI) de Sun. Utilizando uno de ellos, es posible mantener gran parte de la riqueza y precisión que se utiliza con un modelo de programación local, pero que son servidos en sistemas remotos.

El problema con estos sistemas es que no escalan a Internet. Su dependencia en acoplar firmemente al consumidor de los servicios con el servicio implica una infraestructura homogénea, y a menudo significa que son muy quebradizos. Si la implementación en un lado cambia, el otro lado se interrumpe. No hay nada intrínsecamente equivocado al requerir una infraestructura firmemente acoplada, y muchas aplicaciones han sido desarrolladas exitosamente utilizándolas. Sin embargo, este modelo no escala. A medida que las compañías se adquieren entre sí o los proveedores de tecnologías de información cierran o se cambian a otros modelos, es muy duro garantizar una infraestructura sencilla, unificada. No hay garantía de que el servicio con el que desea hablar en el otro extremo de su conexión tendrá la infraestructura requerida. Es posible que no sepa qué sistema operativo, modelo de objetos o lenguaje de programación utiliza.

Formatos de descripción de datos estándar para servicios Web

Los servicios Web, son por definición poco acoplados. Esto significa que puede cambiar la implementación en cualquier extremo de la conexión sin afectar el otro extremo. Técnicamente, esto se logra utilizando tecnología asíncrona basada en mensajes, con excelente desempeño y utilizando protocolos Web tales como HTTP, y lo más importante, XML para lograr un alcance universal.

La clave para hacer que los servicios Web trabajen a través de Internet y su infraestructura heterogénea es que se acuerde en un formato simple de descripción de datos. Ese formato es XML. Específicamente, los servicios Web requieren XML para resolver cuatro problemas distintos:

- Definir un formato de enlace para comunicación entre procesos. En el nivel más bajo, los sistemas necesitan hablar el mismo lenguaje. En particular, las aplicaciones que se comunican necesitan haber establecido las reglas sobre cómo van a representar los diferentes tipos de datos (por ejemplo, enteros y arreglos) y cómo van a representar los comandos (esto es, lo que se debe hacer con los datos). Asimismo, las aplicaciones necesitan una forma de ampliar su lenguaje, si tienen que hacerlo. Simple Object Access Protocol (SOAP) representa un conjunto común de reglas acerca de cómo se representan y amplían los datos y los comandos.
- Describir servicios. Una vez que las aplicaciones tienen reglas generales sobre cómo representar los tipos de datos y comandos, necesitan una forma específica de describir el conjunto de datos y comandos que aceptarán. Por ejemplo, no es suficiente para una aplicación decidir que acepta enteros; en cierta forma, debe haber una manera de determinar que, dados dos enteros, los multiplicará. Web Service Description Language (WSDL) es una gramática XML que los desarrolladores y herramientas de desarrollo pueden utilizar para representar las capacidades de un servicio Web.
- Descubrir servicios Web. Asimismo, requiere un conjunto de reglas que describan cómo se deberían localizar descripciones de servicios. Por ejemplo, ¿dónde una persona o herramienta buscan de manera predeterminada para descubrir las capacidades de un servicio? El protocolo de búsqueda (DISCO) proporciona un conjunto de reglas que permiten a una persona o herramienta de desarrollo descubrir automáticamente una descripción WSDL de un servicio.
- Registrar centralizadamente los servicios Web. La iniciativa Universal Description, Discovery and Integration (UDDI), ayuda a proporcionar un directorio detallado de negocios operando en el mundo digital enumerando los servicios basados en Web que ofrecen. Los vendedores participarán sin costo en UDDI Universal Business Registry proporcionando detalles de contactos, junto con información de productos y servicios. Asimismo, los compradores podrán buscar en el registro (nuevamente sin costo) y localizar compañías que proporcionen los productos o servicios que requieran.

Una vez que estas capas estén en su lugar, podrá buscar un servicio Web,

instanciarlo como un objeto e integrarlo en sus aplicaciones.

Utilizando estándares abiertos de la industria

Todas las tecnologías relacionadas con servicios Web son estándares abiertos de la industria con amplio soporte, y no son propiedad de Microsoft. En la siguiente lista se presentan los recursos adicionales que se relacionan con cada una de estas tecnologías.

- Para más información acerca de XML, visite:
<http://www.w3.org/XML>
<http://msdn.microsoft.com/xml>
- Para más información acerca de WSDL, visite
<http://msdn.microsoft.com/xml/general/wsdl.asp>
- Para más información acerca de SOAP, visite
<http://www.w3.org/TR/SOAP>
<http://msdn.microsoft.com/soap>
- Para más información acerca de DISCO, visite
<http://msdn.microsoft.com/xml/general/disco.asp>
- Para más información acerca de UDDI, visite
<http://www.uddi.org>
<http://www.microsoft.com/PressPass/features/2000/sept00/09-06uddi.asp>

Una arquitectura tecnológica actualizada y moderna

La Figura 3 muestra la arquitectura básica del entorno .NET.

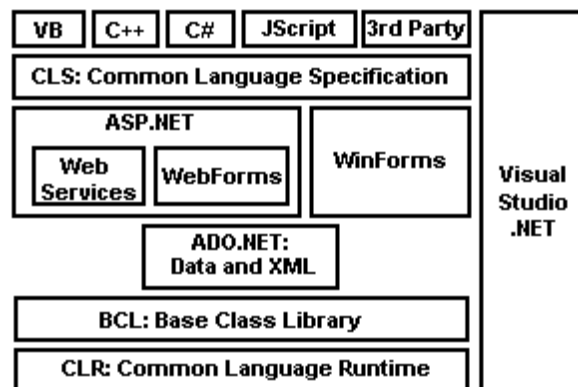


Figura 3: Microsoft .NET Framework

Estos componentes principales del Entorno .NET se describen a continuación:

- El Common Language Runtime (CLR). CLR es el mecanismo de ejecución para las aplicaciones .NET. Proporciona varios servicios, incluyendo la carga y ejecución del código, aislamiento de la memoria de las aplicaciones, administración de memoria, manejo de excepciones, acceso a *metadata* (información mejorada de tipos), y la conversión de MSIL (Lenguaje Intermedio Microsoft) a código nativo.

-
- La Base Class Library (BCL). BCL proporciona un amplio conjunto de clases, lógicamente agrupadas en espacios de nombres jerárquicos que proporcionan acceso a las características más importantes del sistema operativo.
 - ADO.NET. Esta es una actualización evolutiva para la tecnología de acceso a datos ActiveX® Data Objects (ADO) con mejoras importantes destinadas a la naturaleza desconectada del Web.
 - ASP.NET. Esta es una versión avanzada de Active Server Pages (ASP) para el desarrollo de aplicaciones Web (utilizando Formas Web) y desarrollo de servicios Web.
 - La Common Language Specification (CLS). Esta es responsable de hacer que muchas de las tecnologías antes mencionadas estén disponibles para todos los lenguajes que soportan .NET Framework. CLS no es una tecnología, y no hay un código fuente para ella. Define un conjunto de reglas que proporcionan un contrato que rige la interoperabilidad entre los compiladores de lenguaje y las bibliotecas.
 - Win Forms. Este modelo de programación y conjunto de controles proporciona una arquitectura sólida para el desarrollo de aplicaciones basada en Windows.
 - Visual Studio.NET. Este proporciona las herramientas que le permiten explotar las características del Framework para crear aplicaciones concretas.

EL ENTORNO .NET

El CLR simplifica el desarrollo

El Common Language Runtime (CLR) juega un rol tanto en el desarrollo como en la ejecución de aplicaciones .NET. En tiempo de ejecución, CLR es responsable de administrar el entorno de ejecución del código .NET (administrado), incluyendo la asignación de memoria y administración de hilos de ejecución, así como reforzando las políticas de seguridad. Debido a que CLR automatiza estas tareas de desarrollo, el tiempo de desarrollo se reduce y las tareas de desarrollo se simplifican. Por ejemplo, la recolección automatizada de memoria-basura (automated garbage collection) significa que no tendrá que preocuparse acerca de las filtraciones de memoria. Algunos de los componentes clave de CLR se muestran en la Figura 4.

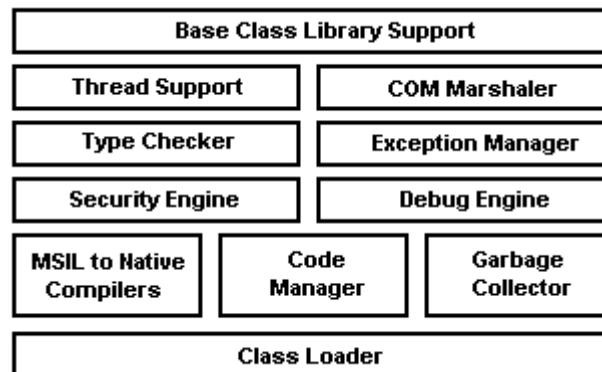


Figura 4: Componentes de CLR

El entorno es multilenguaje y ampliable

Los runtime no son nada nuevo para los lenguajes de programación. El rol crítico del runtime en el entorno .NET (y lo que realmente lo distingue) es que proporciona un ambiente unificado a través de todos los lenguajes de programación.

Las clases base de .NET Framework (proporcionadas dentro de la Biblioteca de clase base o BCL) proporcionan un conjunto unificado de bibliotecas de clases, orientadas a objetos, jerárquicas y ampliables para que la utilicen los desarrolladores. Hoy en día, a medida que un desarrollador utiliza C++ puede utilizar Microsoft Foundation Classes (MFC), como un desarrollador Java puede utilizar Windows Foundation Classes (WFC), y como un desarrollador Visual Basic® utilizaría las API Visual Basic.

.NET Framework unifica los desparejos entornos con los que necesita trabajar hoy en día. Como resultado, no tendrá que aprender múltiples entornos para hacer su trabajo. Como desarrollador de componentes, usted no tiene que decidir en qué lenguaje enfocarse, ya que simplemente puede enfocarse al CLR, sabiendo que su trabajo podrá ser accesible para todos los lenguajes.

Interoperabilidad cruzada entre lenguajes

Entre muchas de las ventajas de .NET se encuentra la transparencia de lenguaje.

Cualquiera que sea el lenguaje administrado que haya elegido, tiene igual acceso a la plataforma. Al crear un conjunto común de APIs a través de todos los lenguajes de programación, .NET Framework le permite cruzar entre lenguajes la herencia, el manejo de errores y la depuración de programas. Como desarrollador, puede aprovechar la plataforma .NET utilizando su lenguaje preferido.

Diseño multi-plataforma

.NET representa una nueva forma de desarrollar software para Windows, y potencialmente, para otras plataformas. .NET no está ligado indivisiblemente al sistema operativo Windows. Por ejemplo, el código intermedio (MSIL) generado por los compiladores de lenguaje .NET no es para un procesador específico, y las clases BCL están diseñadas para trabajar en cualquier sistema operativo.

Asimismo, .NET permite que su aplicación se extienda a múltiples plataformas debido a sus características de interoperabilidad y ampliabilidad. Las aplicaciones .NET están desarrolladas siguiendo CLI (Infraestructura de lenguaje común). CLI consiste de CLR y BCL. Tanto CLI como el lenguaje C# (que se pronuncia “si-yarp”; un lenguaje moderno basado en componentes) han sido enviados al grupo de estandarización internacional ECMA (European Computer Manufacturer's Association) para su estandarización (para más información, visite <http://www.microsoft.com/PressPass/press/2000/Nov00/VSFWbeta1PR.asp>). Microsoft tiene el compromiso de mantener su versión de acuerdo con los estándares ECMA.

La BCL reemplaza las APIs

El .NET Framework proporciona un conjunto común de funcionalidad y servicios en la forma del Base Class Library (BCL). Esta biblioteca ofrece reemplazos para muchas de las APIs Win32® actuales, significando que como desarrollador, usted se enfoca a una plataforma en lugar de a un sistema operativo. Al tiempo que las APIs Win32 siguen estando accesibles a través del Framework, en la mayoría de los casos tendrá la opción de utilizar una clase base. Gracias a que las clases han sido creadas en espacios de nombre bien definidos, es fácil descubrir una clase que podrá ejecutar una tarea específica.

Código administrado y ensambles

Todos los lenguajes .NET se enfocan en CLR y generan “código administrado”. El código administrado se empaqueta, implementa y divide por versiones utilizando los ensambles. Los ensambles se analizan más adelante en la sección “Características de implementación” de este artículo.

Características importantes integradas en el Framework

A continuación se presentan algunas de las características clave proporcionadas por el .NET Framework:

-
- Modelo de programación consistente: Todos los servicios de aplicaciones son accedidos a través de un modelo de programación común orientado a objetos, a diferencia de los actuales donde algunas instalaciones del sistema operativo son accedidas a través de funciones en DLLs, y otras son accedidas a través de objetos COM.
 - Modelo de programación simplificado: .NET busca simplificar en gran medida el canalizado y las misteriosas construcciones requeridas por Win32 y COM. En específico, los desarrolladores ya no tendrán que lograr un entendimiento de la registry, GUIDs, COM's IUnknown interface, mecanismos de control de referencia tales como AddRef y Release, HRESULTS, y demás. Estos conceptos ya no existen en .NET.
 - Ejecutar una vez, ejecutar siempre: Todos los desarrolladores están familiarizados con el Infierno DLL. Ya que al instalar componentes para una nueva aplicación pueden sobreescribirse los componentes de una aplicación anterior, la aplicación anterior puede exhibir un comportamiento extraño o detener su funcionamiento. Ahora, la arquitectura .NET separa los componentes de las aplicaciones de manera que una aplicación siempre carga los componentes en los que fue desarrollada y probada. Si una aplicación se ejecuta después de la instalación, entonces la aplicación siempre deberá ejecutarse.
 - Ejecutarse en varias plataformas: Una vez escrito y compilado para MSIL, una aplicación .NET administrada puede ejecutarse en cualquier plataforma que soporte CLR de .NET (incluyendo plataformas no basadas en Windows). De inmediato se aprecia el valor de este amplio modelo de ejecución, cuando necesita soportar múltiples configuraciones de hardware o sistemas operativos.
 - Integración de lenguaje: Tal como COM permite que diferentes lenguajes de programación interoperen entre sí, .NET permite que los lenguajes se integren unos con otros. Por ejemplo, es posible crear una clase en C# que se derive de una clase implementada en Visual Basic.NET. .NET puede habilitarlo debido a que define y proporciona un Sistema de Tipos Comunes (CTS) para todos los lenguajes .NET. La Especificación de Lenguaje común (CLS) describe qué deben hacer los proveedores de compiladores con el fin de que sus lenguajes se integren con otros lenguajes. Microsoft está proporcionando diversos compiladores que producen código enfocándose en .NET CLR: C++ con ampliaciones administradas, C#, Visual Basic.NET (lo que ahora incluye la categoría Visual Basic Scripting Edition y Visual Basic para aplicaciones (VBA)), y JScript®. Además, compañías diferentes a Microsoft están produciendo compiladores para otros lenguajes que también están destinados a CLR .NET.
 - Reutilización de código: Al utilizar los mecanismos recién descritos, se pueden crear sus propias clases que ofrezcan servicios a aplicaciones de terceros. Esto por supuesto simplifica en gran medida la reutilización de código y amplía el mercado para los proveedores de componentes.
 - Administración automática de recursos a través de la recolección de memoria-basura: la programación requiere grandes habilidades y disciplina. Esto es

especialmente cierto cuando se van a administrar recursos tales como archivos, memoria, conexiones de red, recursos de base de datos, etc. Uno de los tipos más comunes de errores ocurren cuando una aplicación se rehúsa a liberar uno de estos recursos, haciendo que ésta aplicación u otra fallen. CLR de .NET monitorea automáticamente el uso de los recursos, garantizando que su aplicación nunca tendrá escasez de recursos. Ésto se denomina recolección automática de memoria-basura. No hay manera de liberar explícitamente un recurso de memoria, ya que la memoria-basura se recolecta automáticamente cuando ya no es necesaria.

- Seguridad de tipos: CLR de .NET puede verificar que todos su código sea de tipo seguro. La seguridad de tipo asegura que los objetos asignados siempre podrán ser accedidos en formas compatibles. Por esto, si un parámetro de entrada a un método se declara como aceptando un valor de 4-bytes, CLR detecta y monitorea los intentos para pasar a un valor de 8-bytes utilizando este parámetro. Similarmente, si un objeto ocupa 10 bytes en la memoria, la aplicación no puede coaccionar esto dentro de un formulario que permita leer más de 10 bytes. La seguridad de tipo también significa que el flujo de ejecución únicamente se transferirá a ubicaciones bien conocidas (llamados puntos de entrada a métodos). No hay forma de construir una referencia arbitraria a una ubicación de memoria y causar que el código en esa ubicación comience a ejecutarse. La seguridad de tipos y la falta de manipulación directa de índices, sienta las bases del modelo de seguridad, y también elimina muchos errores comunes de programación y ataques clásicos al sistema tales como la explotación de sobrecarga de *buffer*.
- Rico soporte para *debugging*: Debido a que el CLR de .NET es la base común para los lenguajes, ahora es mucho más fácil implementar componentes en su aplicación desarrollada utilizando el lenguaje que mejor se adapte a cada problema. CLR de .NET soporta por completo *debugging* a través de componentes escritos en múltiples lenguajes. Por ejemplo, puede ir al código fuente de un control escrito en C# desde el *debugging* de un programa Visual Basic.NET.
- Manejo consistente de errores: Uno de los aspectos más frustrantes de la programación en la plataforma Windows es la forma inconsistente en la que se reportan errores. Algunas funciones retornan códigos de error Win32, algunas retornan HRESULTS, y algunas originan excepciones. En .NET, todos los errores se reportan a través de excepciones. La ejecución normal del programa se detiene cuando ocurre una excepción, y un adecuado controlador de excepciones entra en ejecución. Asimismo, las excepciones permiten aislar el código de manejo de errores de la lógica del programa. Esto simplifica en gran medida la escritura, lectura y mantenimiento de código. Además, las excepciones trabajan a través de los módulos y límites de lenguajes.
- Implementación: Hoy en día, las aplicaciones basadas en Windows pueden ser increíblemente difíciles de instalar e implementar. Por lo regular existen múltiples archivos, configuraciones de registro y accesos rápidos que tienen que ser creados. Además, desinstalar por completo una aplicación es casi

imposible. Con Windows 2000, Microsoft presenta Windows Installer, un nuevo mecanismo de instalación que ayuda en estos problemas; pero sigue siendo posible que la compañía que crea el Installer Package no lo realice correctamente. .NET busca solucionar por completo estos problemas. Los componentes .NET no están referenciados en el registro, y al instalar la mayoría de las aplicaciones basadas en .NET no se requiere más que copiar los archivos al directorio. Desinstalar una aplicación .NET es tan sencillo como borrar estos archivos. Asimismo, CLR permite implementaciones de lado a lado donde, por ejemplo, se pueden tener dos versiones de la misma aplicación ejecutándose al mismo tiempo en la misma máquina mientras sus usuarios son migrados.

- Seguridad: La seguridad tradicional del sistema operativo proporciona aislamiento y control de acceso basado en cuentas del usuario. Esto ha probado ser un modelo útil, pero se asume que todos el código es igualmente digno de confianza. Esta suposición fue justificada cuando se instala código desde medios físicos (tales como CD-ROM) o servidores corporativos confiables. Pero con el aumento de código móvil, tales como *scripts* Web, las descargas de aplicaciones desde Internet y archivos adjuntos de e-mail, existe la necesidad de un control más granular de comportamiento de las aplicaciones. El modelo Seguridad .NET de Acceso de Código ofrece este control.

Con frecuencia las aplicaciones necesitan forzar comportamientos basados en el concepto de los roles, en contraste a cuentas individuales. Este concepto de rol fue ofrecido por primera vez en Microsoft Transaction Server (MTS), y se proporcionaron más mejoras con COM+. .NET soporta la implementación de roles definidos por la aplicación y control de acceso con base en estos roles. Estos mecanismos son ampliados de manera que son adecuados para ambientes de Internet y otros en general.

MODERNA TECNOLOGIA DE COMPILACIÓN

El MSIL hace que su código sea independiente de la CPU y del sistema operativo

Todos los compiladores de lenguaje .NET emiten Microsoft Intermediate Language (MSIL) enfocado al CLR. MSIL es un lenguaje intermedio independiente de CPU creado por Microsoft después de consultar a diversos escritores de lenguajes y compiladores comerciales y académicos externos. MSIL es un lenguaje de nivel más elevado que los lenguajes de máquina. Entiende los tipos de objeto y tiene instrucciones que crean e inicializan objetos, llamados a métodos virtuales en objetos, manipulan directamente elementos de arrays, e incluso tienen instrucciones que lanzan y capturan las excepciones de manejo de error.

Lo importante que hay que tomar en cuenta acerca de MSIL es que no se apeg a ninguna plataforma de CPU específica. Esto significa que un archivo PE (Ejecutable portátil) que contenga MSIL puede ejecutarse en cualquier plataforma de CPU mientras que el sistema operativo que se ejecuta en esa plataforma albergue el CLR de .NET. Inicialmente, Microsoft planea ofrecer el CLR .NET en Windows 98, Windows NT 4.0 y Windows 2000.

Otro beneficio de MSIL es que proporciona un nivel de abstracción de hardware. Al tiempo que se planea que la primera versión de CLR se ejecute únicamente en plataformas de 32 bits Windows, el desarrollo de una aplicación utilizando MSIL permite que la aplicación sea más independiente del sistema operativo. Por ejemplo, MSIL ayuda a hacer portátil al código a versiones de 64 bits de Windows cuando CLR está disponible en esa plataforma.

Por último, MSIL es vital para la seguridad .NET, ya que la verificación (analizada más adelante) es capaz de examinar la intención del código sin importar el lenguaje de alto nivel que fue utilizado para desarrollarlo.

MSIL es administrado por CLR

Las instrucciones MSIL no se pueden ser ejecutadas directamente por el CPU, por lo que el CLR debe compilar primero las instrucciones MSIL en instrucciones CPU nativas. CLR proporciona a MSIL el acceso a las características del .NET Framework.

La necesidad de compilar MSIL en código nativo hace surgir la pregunta de si CLR debe convertir todos el código MSIL del programa a instrucciones CPU al momento de la carga. La ventaja de hacer esto es que el programa se ejecuta muy rápido. No obstante, la desventaja es que la compilación de MSIL tarda, lo que puede aumentar mucho el tiempo de inicio de la aplicación. Además, es raro que un usuario haga que una aplicación ejecute todo su código. Si CLR compilara todas las instrucciones MSIL es probable que se desperdicie memoria y tiempo. Para resolver este problema, CLR emplea la compilación Justo a tiempo.

Compilación justo a tiempo mejorada

CLR compila las instrucciones MSIL a medida que las funciones son llamadas.

Cuando CLR carga un tipo de clase, conecta código puente a cada método expuesto por la clase. Cuando se solicita el método, el código puente direcciona la ejecución del programa al componente del mecanismo CLR que es responsable de compilar MSIL del método a código nativo. Ya que MSIL ha sido compilado *justo a tiempo* (JIT), este componente de CLR se refiere frecuentemente como un compilador JIT. Una vez que el compilador JIT ha compilado MSIL, el puente del método es reemplazado por la dirección al código compilado. En el momento en que este método sea solicitado en el futuro, únicamente el código nativo se ejecutará y el compilador JIT no se involucrará en el proceso. Como pueden imaginar, esto dispara considerablemente el desempeño comparado con un interprete en tiempo de ejecución estándar.

Además, el compilador JIT soporta descarte de código. Descarte de código es la capacidad de CLR de descartar un bloque de código nativo de un método no utilizado, liberando la memoria ocupada por los métodos que no han sido ejecutados por algún tiempo. Evidentemente, cuando CLR descarta un bloque de código, reemplaza el método con un puente de manera que el compilador JIT pueda acceder el código nativo cuando el método es llamado subsecuentemente.

Cuando el compilador JIT compila el código MSIL en tiempo de ejecución, sabe más acerca del ambiente de ejecución que el compilador. Por ejemplo, el compilador JIT puede detectar que la CPU es una Pentium III y generar instrucciones de CPU que aprovechan las mejoras del desempeño que Intel ha realizado para Pentium III en comparación con Pentium II o Pentium.

Además, el compilador JIT puede generar código que utilice registros de CPU que un compilador evitaría usar normalmente. Asimismo, un compilador JIT puede detectar una variable que siempre contiene un valor específico y como resultado puede generar un código pequeño y rápido que trabaja únicamente debido a que el compilador JIT puede realizar una suposición en tiempo de ejecución.

La Pre-Compilación dispara el desempeño

Otra característica denominada PreJIT proporciona una manera de compilar todo un ensamble a código nativo y guardar los resultados en disco la primera vez que se ejecuta. Cuando se carga el ensamble por segunda vez, se carga la versión pre-compilada guardada. Como resultado, la aplicación se carga mucho más rápido.

Al instalar .NET Beta SDK en su máquina, la instalación pre-compila silenciosamente algunos ensambles del sistema, tales como MSCORLIB, los cuales contienen varias clases del sistema principal, requeridas por todas las aplicaciones .NET. Este proceso analiza el código binario MSIL del ensamble y su *metadata*, calcula información del esquema de clases, y compila los métodos en su equivalente de código nativo.

Futuras versiones de la Beta SDK harán la misma tarea, pero con más ensambles del sistema. Además, podrá compilar previamente sus propios ensambles

utilizando una herramienta de línea de comandos denominada NGEN (Generador de Código Nativo).

Después de ejecutar NGEN en un ensamble, el archivo de salida se guarda en el Global Assembly Cache (GAC) en el disco duro local. Utilizar esta técnica para compilar previamente los ensambles resulta en un mejor tiempo de carga del ensamble, y mejor desempeño en tiempo de ejecución, debido a que no hay necesidad de que JIT compile cada método de MSIL en código nativo cuando sea referenciado.

Actualizando componentes dinámicamente

NGEN registra información de la configuración en el archivo de salida previamente compilado. Si los detalles de la configuración cambian entre el momento en que se ejecutó NGEN, y el momento en que ejecuta la aplicación (por ejemplo, una nueva versión del ensamble dependiente se instala en la máquina; o cambia la política de seguridad) entonces CLR detectará automáticamente este cambio y revertirá a utilizar el compilador JIT regular. De esta forma, puede reemplazar ensambles con nuevas versiones en el momento de ejecución, sin caer en problemas ocasionados por DLLs bloqueados. En la figura 5 se muestra una descripción general del proceso de compilación JIT.

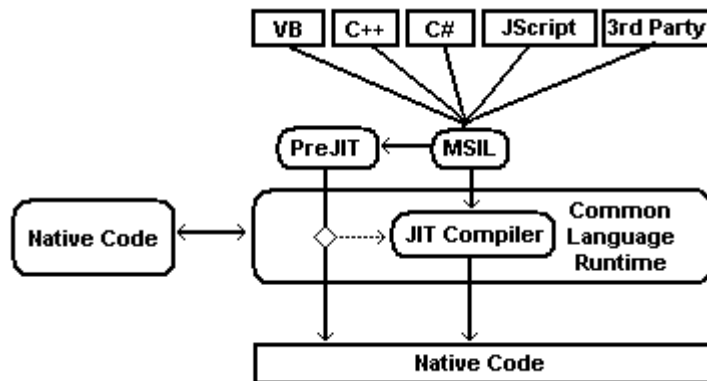


Figura. 5: Los compiladores de lenguaje generan MSIL, los cuales pueden ser compilados al momento de la instalación (con PreJIT), o utilizando el compilador JIT al momento de la ejecución. CLR detecta cualquier cambio que requiera compilación.

Los espacios de nombre proporcionan un paradigma de programación consistente

La Biblioteca de clases base (BCL) se incluye en .NET como un ensamble de entorno base que contiene varias definiciones de clase, donde cada clase expone una característica descrita en la plataforma. Debido a que Microsoft define cientos de clases en BCL, la biblioteca se divide en espacios de nombre que agrupan de manera lógica las clases relacionadas.

Por ejemplo, el espacio de nombre System contiene la clase base Object, desde el cual se derivan todas las demás clases. Además, el espacio de nombre System

contiene clases para manejo de excepciones , recolección de memoria-basura, de consola de entrada/salida, así como una variedad de clases de utilidades que realizan conversiones seguras de tipos, formatean tipos de datos, generan números aleatorios y realizan varias funciones matemáticas.

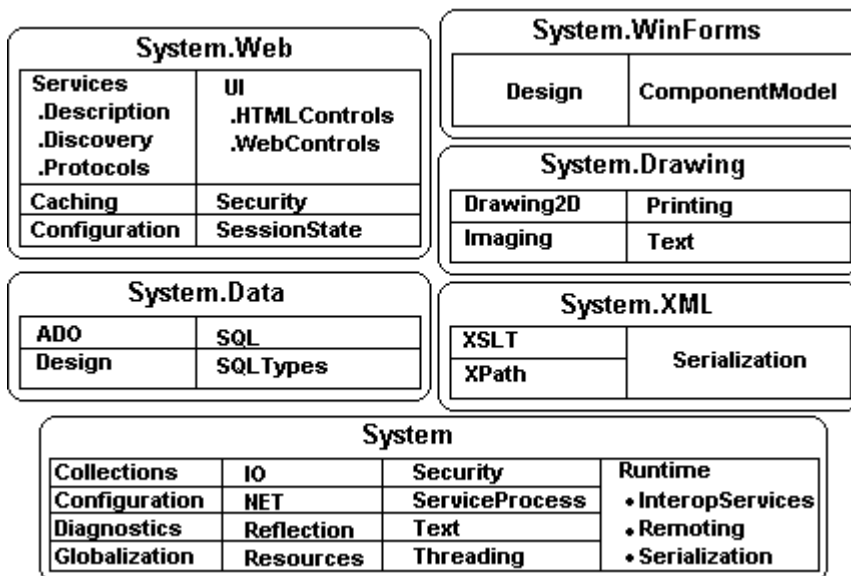


Figura. 6: Espacios de nombre de la BCL

Para acceder a cualquiera de las características de la plataforma, necesita trabajar con las clases relevantes definidas dentro de estos espacios de nombre. Si desea personalizar el comportamiento de cualquier clase proporcionada por el sistema, puede derivar su propia clase de él.

El .NET Framework es ampliable

.NET puede presentar un paradigma de programación consistente para los desarrolladores de software debido a la naturaleza orientada a objetos de la plataforma. Los desarrolladores pueden crear sus propios espacios de nombre que contengan sus propias clases. Esto se enlaza sin problemas con las clases existentes, simplificando por mucho el desarrollo de software en comparación con los enfoques clásicos de programación basados en Windows.

EL COMMON TYPE SYSTEM

CLR implementa un sistema de tipos genéricos

La especificación formal del sistema de tipos implementada por CLR se denomina Common Type System (CTS). CTS especifica cómo se definen las clases de objetos (denominadas tipos). Por ejemplo, CTS permite que un tipo de clase contenga cero o más miembros. Los miembros posibles incluyen:

- **Campo:** una variable de datos que forma parte del estado interno del objeto. Los campos son identificados según su nombre y tipo.
- **Método:** una función que realiza una operación en el objeto, por lo regular cambiando el estado del objeto. Los métodos tienen un nombre, firma y modificadores. La firma especifica la convención a quien lo invoca, número de parámetros (y su secuencia), los tipos de parámetros, y el tipo de valor retornado por el método. Los modificadores pueden incluir si es un método público, privado, estático y demás.
- **Propiedad:** tradicionalmente, las propiedades se han visto de manera muy semejante a los campos y han sido simuladas utilizando llamadas de métodos COM. Lenguajes tales como Visual C++ tradicionalmente han expuesto los mecanismos internos (por ejemplo a través de los atributos de Lenguaje de Definición de Interfaz), mientras lenguajes de nivel más alto tales como Visual Basic han mantenido ocultos esos mecanismos. Las propiedades son extremadamente útiles en la medida que permiten a un desarrollador de clases calcular un valor únicamente cuando es necesario y permiten al usuario de la clase acceder al valor con una sintaxis simple. Las propiedades son como “campos inteligentes” y permiten que los datos sean procesados personalmente cuando se se le toma o define el valor. Las propiedades de las clases .NET ahora son ciudadanos de primera clase y se definen utilizando sintaxis de lenguaje específica. Por ejemplo, el siguiente código C# muestra una clase denominada Button, exponiendo la propiedad denominada Caption:

```
class Button
{
    public string Caption // property
    {
        get { return caption; }
        set { caption = value; }
    }
    private string caption; // field
}
```

- **Evento:** los eventos proporcionan un mecanismo de notificación entre un objeto y otros objetos interesados. Por ejemplo, un botón podría ofrecer un evento que notifique otros objetos cuando el botón se oprime.

Límites de visibilidad flexibles para tipos

Asimismo, CTS especifica las reglas de visibilidad de los tipos y el acceso a los

miembros de un tipo. Los tipos son visibles desde fuera del ensamble, (es decir para los clientes del ensamble), o son visibles únicamente para codificarse dentro del mismo ensamble. Al marcar un tipo como público se permite que sea visible (exportado) fuera del ensamble. Con esto, CTS establece las reglas mediante las cuales los ensambles forman un límite de visibilidad (un alcance) para un tipo y sus métodos, y Common Language Runtime (CLR) fuerza las reglas de visibilidad. Sin importar si el tipo está visible para quien lo invoca, el tipo tiene el control de si quien lo invoca accedida a sus miembros. Las opciones válidas para controlar el acceso a un método o campo son:

Access Type	Description
Public	The method is callable by any code in any assembly.
Private	The method is callable only by other methods in the same class type.
Family	The method is callable by derived types, regardless of whether they are within the same assembly.
Assembly	The method is callable by any code in the same assembly.
Family and Assembly	The method is callable by derived types only if the derived type is defined in the same assembly.
Family or Assembly	The method is callable by derived types in any assembly. The method is also callable by any types in the same assembly.

Comportamiento consistente de los objetos

De igual forma, CTS define las reglas que rigen el comportamiento de un objeto, incluyendo la herencia de tipo, funciones virtuales y tiempo de vida del objeto. Estas reglas han sido diseñadas para acomodar semánticas expresables en los lenguajes que se utilizan hoy en día. De hecho no tiene que aprender las reglas del CTS *per se* , ya que el lenguaje que elige expone su propia sintaxis de lenguaje y reglas de tipo, y correlaciona esta sintaxis específica del lenguaje en una sintaxis de CTS cuando emite el archivo MSIL. Por ejemplo, C# utiliza el modificador Protected para representar el tipo de acceso Family.

Es mejor pensar en el lenguaje y el comportamiento de su código como dos cosas separadas y distintas. Al utilizar Visual C++, pueden definir sus propias clases con sus propios miembros. Por supuesto, podría haber utilizado C# o Visual Basic.NET para definir la misma clase con los mismos miembros. Al tiempo que la sintaxis que utiliza para definir esta clase difiere del lenguaje que elija, el comportamiento de la clase es absolutamente idéntico debido a que CTS del CLR define el comportamiento de la clase.

Herencia simple de clase, herencia múltiple de interfaces

CTS soporta la herencia simple de implementación, y herencia múltiple de interfaces. Asimismo, CTS obliga a que todos los tipos de clase deben derivarse

de un tipo de clase predefinido `System.Object` (donde `Object` es un nombre de tipo dentro del espacio de nombre `System`). `Object` es la raíz de todos los demás tipos de clase y por lo tanto garantiza que cada tipo de clase tenga el conjunto mínimo de comportamientos. Específicamente, el tipo `System.Object`:

- Permite que dos objetos sean comparados en cuanto a igualdad
- Permite identificar un objeto con un *hash code*
- Permite consultar el tipo de ejecución del objeto
- Permite realizar una copia superficial (*bitwise*) del objeto
- Permite obtener una representación en cadena del estado actual del objeto.

Integración e interoperación multilenguaje

COM permite a los objetos creados en diferentes lenguajes comunicarse entre sí, pero únicamente a través de la interfaz COM. Por el contrario, CLR de .NET integra todos los lenguajes y permite objetos creados en un lenguaje ser tratados como ciudadanos en código escrito en un lenguaje completamente diferente. CLR hace esto posible debido a su conjunto de tipos estándar, información de tipo autodescriptivo (denominada *metadata*), y un ambiente de ejecución común.

CLS como subconjunto de CTS

Si pretende crear tipos .NET que sean fácilmente accesibles desde otros lenguajes de programación, es importante utilizar características de su lenguaje de programación que estén garantizadas para estar disponibles en todos los demás lenguajes. Para ayudarlo con esto, Common Language Specification (CLS) informa a los proveedores de compiladores sobre el conjunto mínimo de características que los compiladores deben soportar para CLR. Diversos proveedores ya trabajan con versiones compatibles con .NET de sus compiladores.

Observe que CTS soporta muchas más características que el subconjunto definido por CLS, por lo que si no se preocupa acerca de la operabilidad entre los lenguajes puede desarrollar tipos de clase muy ricos que son limitados únicamente por el conjunto de características del lenguaje.

Todos los lenguajes son iguales

En lo que se refiere a CLR de .NET, todos los lenguajes administrados son iguales, debido a que se basan en el mismo CLS (Common Language Specification), y finalmente, el mismo CTS (Common Type System).

Al tiempo que algunos lenguajes ofrecen ciertas características que otros no, estas son características de lenguaje, no características CLS. Por ejemplo, Visual Basic trata todas las variables y nombres de función como símbolos no sensitivos a mayúsculas y minúsculas.

El código administrado tiene un amplio soporte en la industria

Microsoft proporcionará dos compiladores para producir el código administrado (Visual Basic.NET y C#), así como extensiones para que C++ produzca un código administrado (Extensiones administradas para C++).

Además de Microsoft, muchas compañías están produciendo compiladores que generan código administrado. Los lenguajes adicionales incluyen: Cobol, Haskell, Mercury, ML, Component Pascal, Perl, Python, Eiffel y Smalltalk.

INTRODUCCION A LA SEGURIDAD E IMPLEMENTACION

Los sistemas de Seguridad e Implementación trabajan en conjunto para ayudar a reducir la probabilidad de que código no administrado pueda ejecutarse, y reducir los retos de implementación de la aplicación.

Un sistema de seguridad granular y basado en evidencias .NET proporciona un sistema de seguridad extremadamente granular, permitiéndole controlar si se debe permitir ejecutarse a módulos de código específicos, y de ser así, con qué privilegios. Esto incluye el código descargado a través de Internet.

Por ejemplo, cuando se descarga una aplicación para el entorno.NET, ésta pide un conjunto de permisos (como el permiso de escribir en un directorio temporal). CLR reúne la evidencia acerca de esa aplicación, como por ejemplo: dónde fue descargada, si tiene la firma Authenticode™, o incluso qué está tratando de acceder en el sistema. Con esta información, trabaja administrativamente con un política definida para determinar si la aplicación se debe ejecutar o no. CLR puede indicar a la aplicación que no puede otorgar todos los permisos solicitados, y dar a la aplicación la opción, si desea continuar ejecutándose o no.

Implementación de lado a lado

Con un sistema implementado de seguridad basado en evidencias, se reducen en gran medida los problemas de implementación de la aplicación. Uno de los retos más grandes que enfrentan los desarrolladores y administradores (y por último los usuarios) es el versionamiento. Si sus aplicaciones trabajan un día y no al siguiente debido a la instalación de una nueva aplicación no relacionada, es muy frecuente que en esta situación la nueva aplicación halla sobrescrito una de las bibliotecas compartidas y (con más frecuencia) solucionado un *bug*, cambiando algún comportamiento en los que se respaldaban las aplicaciones existentes. Esta ocurrencia es conocida como el Infierno de los DLL.

El Entorno.NET incluye avances que virtualmente eliminan este problema. En primer lugar, incluye un sólido sistema interno de nombres que proporciona distintos espacios de nombre. Esto significa que no hay forma de que dos bibliotecas que hasta pueden tener el mismo nombre de archivo, se confundan una con la otra. Asimismo, existe una nueva característica denominada implementación "de lado a lado". Si la nueva aplicación en el ejemplo anterior realmente sobrescribe una biblioteca compartida, la aplicación existente puede repararla. La siguiente vez que se inicie la aplicación; verifica cada uno de los archivos compartidos. Si uno ha cambiado y los cambios son incompatibles, puede pedir a CLR que busque una versión con la que sabe que se puede trabajar. Debido a los sistemas de seguridad y versionamiento, CLR lo puede hacer de manera segura.

La seguridad es integral al diseño de .NET.

El chequeo de verificación comienza tan pronto como se carga la clase. Después

se amplía a los recursos de acceso del código a través de seguridad de acceso del código. Asimismo, penetra la arquitectura CLR para asegurar que se cumplan los límites de aislamiento de las aplicaciones.

Estos mecanismos trabajan ampliando los mecanismos de seguridad comúnmente encontrados en los sistemas operativos actuales. Sobre todo, la seguridad del Entorno .NET incluye las siguientes áreas:

- Seguridad de tipo: los programas de tipos seguros hacen referencia únicamente a la memoria que ha sido asignada para su uso, y acceden a objetos únicamente a través de sus interfaces expuestas. Desde el punto de vista de seguridad, al hacer referencia únicamente a la memoria designada se permite que múltiples objetos compartan de manera segura un espacio de dirección único. Al acceder a los objetos únicamente a través de sus interfaces expuestas garantiza que las revisiones de seguridad relacionadas con interfaces específicas no sean evitadas.
CLR asegura la seguridad de tipos combinando la que la definición de tipos esté en *metadata* (parámetros, miembros y elementos de arreglos, valores de retorno de métodos y valores estáticos) con sólida escritura en MSIL (variables locales y asignaciones de pila). Esto sienta las bases para una verificación automática de la seguridad de tipo de los programas compilados en MSIL, sin importar el lenguaje que lo produce. Asimismo, dependiendo del código que sea cargado por CLR .NET, requiere la verificación antes de permitir ejecutarse.
- Evidencia: existen únicamente dos formas para que un código sea ejecutable. La primera, a través del cargador de clases, la segunda a través de los servicios de interoperabilidad (más acerca de esto más adelante). Estos dos servicios son proporcionados por CLR y son parte del perímetro de seguridad. Por ejemplo, el cargador de clases mantiene información acerca de la fuente para cada implementación que carga. Por lo tanto, el cargador de clases puede proporcionar de manera confiable alguna evidencia sobre la que basar la identidad del código. La evidencia puede incluir información como zona de Internet y sitio del que el código fue originado, su nombre compartido, y la identidad del publicador. Utilizando esta información, la política de seguridad puede controlar los privilegios proveídos a una aplicación específica o conjunto de aplicaciones relacionadas.
- Seguridad de acceso de código: este mecanismo proporciona una manera de obligar a la seguridad basado en políticas sobre privilegios otorgados para ejecutar ensamblados. Estos privilegios pueden controlar el acceso a los recursos del sistema y otros códigos, con base en la noción de los permisos. .NET proporciona un amplio conjunto de permisos relacionados con recursos comunes e identidad de código basada en evidencias. Las herramientas o desarrolladores de aplicaciones pueden ampliarlos agregando nuevos tipos de permiso para cumplir con necesidades específicas. Los permisos se agrupan de manera local en conjuntos de permisos que son otorgados para los ensamblados por parte del sistema de seguridad .NET. Estos permisos

otorgados definen lo que el código puede hacer. Se determinan con base en la evidencia del código. La evidencia se utiliza para determinar un conjunto de políticas de código para un ensamble determinado, el cual es utilizado por el sistema de seguridad para determinar el conjunto de permisos que le va a otorgar.

Por lo regular, los permisos son demandados por un código confiable (por ejemplo una clase) que encapsula el acceso al sistema de archivos local. Una demanda hace que el sistema de seguridad verifique que los ensambles que llaman tengan el permiso requerido. La mayoría de las aplicaciones simplemente harán llamadas a dicho código de administración de recursos confiables y no necesitarán incorporar ningún código de seguridad específico.

- **Permisos de recurso:** estos permisos están relacionados con la información o recursos de comunicaciones dentro del sistema. Los ejemplos incluyen archivos, ventanas, el portapapeles, canales de red, almacenamiento privado de aplicaciones y la capacidad de hacer llamadas a APIs no administradas. (Almacenamiento privado de aplicaciones es proporcionado por la clase `IsolatedStorageFile`, un nuevo conjunto de tipos y métodos soportados por la plataforma .NET para el código que no tiene derechos de acceso a archivos). Estos tipos de permiso se pueden utilizar para determinar si el código tiene derecho de acceder a recursos específicos ya sea en tiempo de carga o de ejecución. Esto se relaciona muy de cerca con la Seguridad declarativa, la que será analizada en breve.

Considere una aplicación que desea leer y escribir un archivo en `C:\Temp`. Para abrir un archivo en este directorio, una aplicación administrada utilizaría la clase `System.IO.File`. Esta clase contiene un método estático denominado `Open`. De manera interna, el método `Open` toma el nombre de ruta del archivo que la aplicación desea abrir, y pregunta al mecanismo CLR si está permitido el acceso al nombre de ruta específico. Esto se llama acceso por demanda. En este caso, el método `Open` solicita que al código de la aplicación le sea otorgado un permiso `FileIO` con acceso de lectura y escritura a `C:\Temp\*.*` antes de intentar abrir el archivo. Cuando un método demanda el acceso, CLR verifica la historia de llamadas para ver si todos los ensambles en la cadena de llamada tienen el permiso. Si a algún código de la cadena de llamada le falta el permiso requerido, entonces la demanda falla y se lanza una excepción de seguridad, de otra forma, se llevaría a cabo la demanda. Para completar la operación `File.Open`, la clase `File` necesita llamar a las APIs del sistema de archivos proporcionadas por el sistema operativo. Esto requiere el nuevo permiso `UnmanagedCode`, el cual es, por lo regular, otorgado a clases de administración de recursos altamente confiables. El sistema verifica que la clase de `File` tenga permiso antes de permitir llamar a un código no administrado. Los otros ensambles semiconfiables en la cadena de llamada se espera que no tengan estos permisos. Para permitir que un permiso `UnmanagedCode` sea exitoso, se permite que el código altamente confiable mantenga su permiso de “derecho de utilizar”. Esto hace que el avance en la pila de verificación de permisos termine en el ensamble que alega el permiso.

-
- Permisos de identificación: estos permisos se basan en la evidencia relacionada con un ensamble determinado (sitio, zona, nombre compartido y publicador). Le permiten controlar el acceso a métodos de clase basados en esta información. Los permisos de identificación operan de la misma forma que los permisos a recursos. Si los llamantes tienen el atributo de identidad requerido, la llamada tiene éxito; de otra forma se lanza una excepción de seguridad.
 - Seguridad declarativa: este poderoso mecanismo le permite insertar verificaciones de seguridad de acceso de código en su código declarando las clases, campos y métodos. Estas declaraciones son codificadas en *metadata* del ensamble y son forzadas por el sistema de seguridad .NET. Al utilizar estas declaraciones, puede solicitar que se realicen verificaciones ya sea en tiempo de carga o de ejecución. Aunque son más limitadas, las verificaciones de tiempo de carga son más eficaces en que sólo son revisadas una vez durante la ejecución, como opuesto a que sean revisadas en cada llamada a método. Únicamente verifica los permisos (recurso o identidad) del llamante inmediato, por lo que necesita evaluar cuidadosamente si es adecuado para cada situación. Por lo regular, las revisiones de tiempo de carga se utilizan para limitar el acceso a clases específicas. Por ejemplo, puede utilizar una revisión de identidad en tiempo de carga para asegurar que únicamente el código de un publicador específico o con una clave pública de nombre compartido específica, puedan ser los llamantes inmediatos para la clase. Asimismo, puede utilizar estas verificaciones para limitar la capacidad de crear subclases o sobrescribir métodos virtuales.

Las verificaciones declarativas en tiempo de ejecución son medios simples que sirven para indicar pedidos de permisos y derechos para tipos determinados. Los pedidos ocasionan que se recorra la pila completa de llamadas para verificar los permisos de todos el código de la cadena de llamadas. Son apropiados cuando solicitan una acción y atributos de permiso estáticos. Por ejemplo, si siempre demanda permiso de lectura para C:\Temp es fácil expresar esto de manera declarativa. Pero si necesita crear dinámicamente las validaciones para cubrir diferentes modos de acceso, nombres de directorio y de archivos; entonces se necesitan verificaciones imperativas.
 - Seguridad imperativa: usted implementa estas revisiones de manera programática dentro de un método. Son forzados exactamente de la misma forma que las revisiones en tiempo de ejecución declarativas. El beneficio de utilizarlas es que puede soportar la determinación dinámica de permisos específicos que son necesarios en un contexto de ejecución determinado. Esto es típicamente importante para permisos tales como FileIO, e IsolatedStorage, los cuales soportan múltiples modos de acceso y sub-agrupamiento de los recursos que son accedidos.
 - Seguridad basada en políticas: permitir el uso efectivo de seguridad de acceso de código requiere un sistema basado en políticas. La política le permite declarar específicamente lo que puede permitirse al código, con base en su punto de origen y otra información de identificación. Al utilizar esta política,

automáticamente se otorgan los privilegios correspondientes para el código sin requerir interacción alguna con el usuario. Es muy probable que .NET incluya una política predeterminada que refleja los diferentes niveles de confianza que comúnmente le coloca a su código de sistema local, al código de servidores confiables conocidos, y al código de sitios de Internet no confiables. Podrá refinar esta política si es necesario.

- Seguridad basada en rol: es común para los desarrolladores tomar decisiones de autorización basadas en la identidad o roles asociados con el contexto de ejecución. Con frecuencia se ve esto en sistemas financieros o de negocios como medios para forzar políticas. Por ejemplo, pueden poner límites en el valor de una transacción, dependiendo del rol del usuario que hace la solicitud. Los empleados deben poder procesar transacciones hasta un valor determinado, los supervisores hasta un nivel más alto, y los gerentes en un nivel aún más alto. .NET proporciona servicios para permitir que las aplicaciones incorporen fácilmente dicha lógica de manera que sea independiente de la plataforma y escalable para Internet.

Los mecanismos .NET se desarrollan alrededor de la noción de identidades y principales. Las identidades encapsulan nombres de identidades que entiende la aplicación. Esto puede relacionarse con la noción de cuentas del sistema operativo, pero también pueden ser definidos con la aplicación. El correspondiente principal encapsula la identidad, junto con la información del rol correspondiente. Esto puede o no relacionarse con una noción del sistema operativo como los grupos.

En operación, un proveedor de autenticación confiable típicamente computa la información principal con base en las credenciales relacionadas con una petición determinada (HTTP Put, llamada de procedimiento remota, y demás). Así pues, el principal se adjunta al contexto de ejecución, poniéndolo disponible al código de la aplicación para verificar el rol o la información de identidad antes de tomar una acción determinada.

- Seguridad de ejecuciones Remotas: .NET proporciona soporte para la invocación de objetos remotos que pueden expandir AppDomains. AppDomain es una aplicación lógica y múltiples AppDomains se pueden ejecutar en un solo proceso Win32. Al cruzar los límites de la máquina, los aspectos de seguridad tales como autenticación, autorización, confidencialidad e integridad son críticos, de manera que .NET proporciona soporte para estos mecanismos a fin de que sea compatible con los protocolos de red existentes, y los integra firmemente en la infraestructura de ejecuciones remotas. Esto es más simple para incorporar estas características en sus aplicaciones. Los mecanismos de autenticación son adecuados para la identificación de usuarios así como aplicaciones o entidades de negocios. Una infraestructura de identidad basada en claves se proporciona para administrar tales entidades e integrarlas con los protocolos de autenticación basados en claves. Los servicios de confidencialidad e integridad aprovechan las técnicas criptográficas. Es posible utilizar mecanismos de punto a punto tales como Secure Sockets Layer (SSL) e IP Security Protocol (IPSec). Tecnologías de

encriptación de la capa de la aplicación, integridad y de firma digital en el nivel de mensajería también son soportadas por el entorno .NET.

- **Criptografía:** .NET proporciona un conjunto de objetos criptográficos que soportan algoritmos bien conocidos y usos comunes incluyendo partición, encriptación y generación de firmas digitales. Estos objetos están diseñados de manera que facilitan la incorporación de estas capacidades básicas en operaciones mas complejas, tales como la firma y encriptación de un documento. Los objetos criptográficos son utilizados por .NET para soportar servicios internos, pero también están disponibles para los desarrolladores que necesitan soporte criptográfico.

.NET COMO EVOLUCIÓN DE COM

Infraestructura entre componentes invisible

El entorno .NET automatiza los detalles de "la red de conexiones entre componentes" de la escritura de software, permitiéndole enfocarse en la escritura de lógica de negocios en lugar de la infraestructura COM, utilizando cualquier lenguaje.

Una de las metas principales del Entorno .NET es facilitar el desarrollo de componentes. Uno de los aspectos más duros para crear software basado en componentes utilizando COM, es lidiar con la infraestructura COM. En consecuencia, para facilitar el desarrollo de componentes, el entorno .NET automatiza virtualmente todas las complejas tareas internas que tradicionalmente están relacionadas con el desarrollo COM, incluyendo el conteo de referencias, descripción de interfaces y registración.

Los Componentes .NET y componentes COM son iguales

Usted puede invocar un componente .NET desde un componente COM desarrollado utilizando Visual Studio 6, y el componente .NET se verá como un componente COM regular. A la inversa, puede invocar un componente COM desde un componente .NET desarrollado utilizando Visual Studio.NET y el componente COM se verá como un componente .NET.

Existe una advertencia para esta relación. Como desarrollador COM, debe realizar manualmente muchas de las cosas que como desarrollador .NET puede confiar al CLR para que se encargue de automatizarlo. Por ejemplo, para asegurar totalmente un componente COM, debe escribir código manualmente y realizar administración de memoria manualmente. Asimismo, para instalar un componente COM, debe colocar entradas en el registro de Windows. Para los componentes .NET, CLR automatiza estas características. Los componentes son autodescriptivos y puede instalarlos sin tener que registrarlos en el registro de Windows.

.NET y COM+

COM+ se refiere a la versión de COM que combina las características basadas en servicios de Microsoft Transaction Server (MTS), con COM distribuido (DCOM). COM+ proporciona un conjunto de servicios orientados a la capa media. En particular, COM+ proporciona administración de procesos, servicios de sincronización, un modelo de transacción declarativo y agrupación de conexiones a objetos y bases de datos.

Los servicios COM+ están principalmente orientados hacia el desarrollo de la aplicación de capa intermedia y se enfoca en proporcionar confiabilidad y escalabilidad para aplicaciones distribuidas a gran escala. Estos servicios son complementarios a los servicios de programación proporcionados por el Entorno .NET; y la BCL (Base Class Library) proporciona acceso directo a ellos.

Administración automática

El entorno .NET hace progresar las características COM+, lo que resulta en mucha mayor productividad del desarrollador. Por ejemplo, el entorno .NET introduce la administración automática de memoria, sin importar el lenguaje de programación. Asimismo, proporciona un entorno de programación sencillo y unificado que reúne las mejores características de las Windows® Foundation Classes, Microsoft Foundation Classes, y APIs de Visual Basic®, y las aumenta con otras capacidades tal como una avanzada arquitectura de acceso a datos.

De igual forma, el entorno .NET utiliza los mismos servicios de componentes de bajo nivel que COM+ proporciona hoy en día, con características tales como transacciones, puesta en cola (queuing) y agrupación (pooling) de objetos. En próximas versiones, también proporcionará una característica llamada particionamiento; que proporcionará aislamiento de procesos más sólido y estará diseñada para los proveedores de servicio de aplicaciones. COM+ es el conjunto de servicios de aplicaciones de más alto desempeño y rico en características disponible hoy en día.

Explotando las inversiones COM existentes

Microsoft le está permitiendo aprovechar su inversión existente en COM. Las aplicaciones pueden beneficiarse a partir de las nuevas características del entorno .NET al tiempo que reutilizan los componentes COM existentes.

Aunque no tiene que cambiar el código existente, el cual seguirá trabajando a la par del código administrado desarrollado utilizando los nuevos lenguajes .NET, algunas características del entorno .NET se comportan bastante diferente. Características tales como la administración automática de memoria del entorno .NET, y sus nuevas características de implementación, son muy diferentes de sus equivalentes en COM tal como el conteo de referencias y mecanismos de registro. Aprovechar al máximo algunas de estas características hará necesario adaptar los componentes COM existentes; pero esto se puede hacer cuando sea más adecuado, según los requerimientos de negocios, tiempos de proyecto y disponibilidad de programadores.

Interoperabilidad con código no administrado

El entorno de código administrado proporcionado por CLR alberga varias características que hacen que el desarrollo de software sea más placentero. No obstante, el sistema operativo en el que se ejecuta CLR también ofrece características por su cuenta, y sería desafortunado si las aplicaciones administradas fueran prevenidas de utilizar los servicios de sistema operativo nativo u otro código no administrado.

Para soportar este requerimiento, CLR ofrece el espacio de nombre System.Runtime.InteropServices. Este espacio de nombre contiene un conjunto de tipos que permiten acceder a código no administrado a aplicaciones administradas. Específicamente, existen tres escenarios soportados de interoperabilidad:

- Código administrado que llama funciones DLL no administradas,
- Código administrado que instancia y llama métodos de interfaz de objetos COM, y
- Código no administrado que instancia y llama métodos en objetos .NET.

Interoperabilidad de .NET con funciones DLL no administradas

Para llamar a una función DLL no administrada, debe mencionar al CLR el nombre de la función que solicita (como MessageBoxA), el nombre de DLL que contiene la función (como User32.dll), y cómo acomodar los parámetros de la función (por ejemplo, tipos de datos de parámetros y que parámetros son de entrada, salida o de entrada/salida).

Interoperabilidad de .NET con COM

Para trabajar con un objeto COM, debe trabajar con un envoltorio .NET de manera que los clientes .NET crean que están llamando código .NET; el CLR maneja los detalles. Una utilidad analiza la biblioteca de tipos COM y produce un DLL administrado cuya metadata describe los métodos que envuelve la interfaz del objeto COM. Ahora, una aplicación administrada puede crear instancias de un objeto y utilizarlas como si fuera un tipo nativo administrado. Para cada objeto COM, CLR genera un envoltorio invocable en tiempo de ejecución (RCW, Runtime Callable Wrapper). Este envoltorio actúa como un intermediario entre el código administrado y no administrado, manejando todas las tareas administrativas tales como ordenamiento de parámetros (marshaling), la administración de ciclos de vida, y demás. La Figura 7, muestra un RCW generado en tiempo de ejecución para un objeto COM.

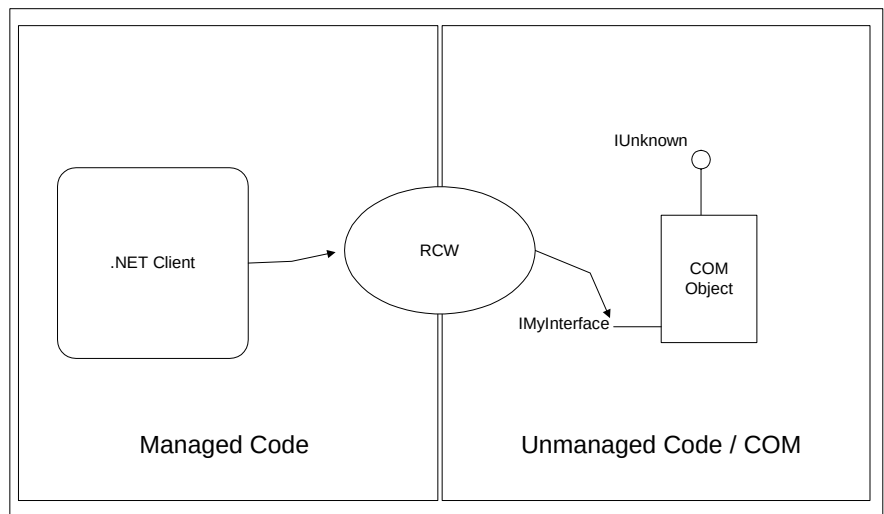


Fig. 7: Un envoltorio .NET invocable en tiempo de ejecución

Interoperabilidad de código no administrado con .NET

Para inicializar y utilizar un objeto administrado .NET desde una aplicación no administrada, debe agregar la definición de la clase .NET al registro del sistema, ya que esta es la forma como COM espera ubicar los objetos en tiempo de ejecución.

El .NET SDK suministra una utilidad de registro de ensambles (*regasm*), que genera una GUID para el objeto .NET y actualiza el registro. Si el código no administrado desea una biblioteca de tipo, se puede generar una. Usted puede crear instancias de objetos desde una aplicación no administrada, y utilizarlos como si fueran un objeto COM. Para cada tipo de objeto .NET administrado, CLR genera un envoltorio invocable COM (CCW: COM Callable Wrapper). Este envoltorio actúa como un intermediario entre el código no administrado y el administrado, controlando todas las tareas administrativas tales como ordenamiento de parámetros, administración de ciclos de vida. La Figura 8 muestra CCW y la interacción entre un componente .NET y un cliente COM.

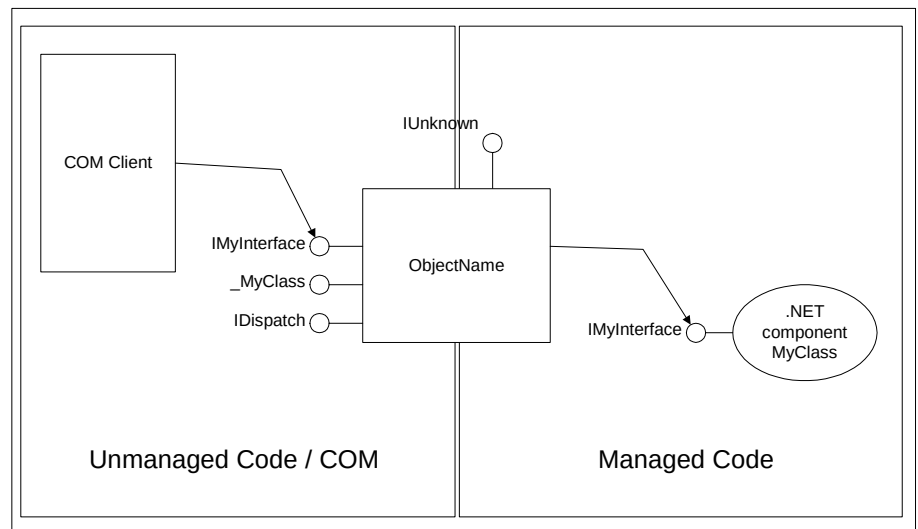


Fig 8. Un envoltorio COM invocable

Debido a las características de interoperabilidad de CLR, no tiene que resignar nada para obtener los beneficios ofrecidos por CLR. Además, el CLR hace que el trabajar con objetos COM sea más fácil que si se hiciera desde código C/C++ no administrado.

La transición entre código administrado y no administrado tiene sus problemas de desempeño, por lo que debería intentar reducir estas transiciones en la medida de lo posible. Esto puede significar portar sus objetos COM a .NET. Sin embargo, las características de interoperabilidad le permiten tomar la decisión cuando sea adecuado hacerlo.

CARACTERISTICAS DE IMPLEMENTACION

La plataforma .NET utiliza metadata y ensambles para almacenar información acerca de los componentes, permitiendo la programación cruzada entre lenguajes y resolviendo los problemas del Infierno de las DLL. Se utiliza *metadata* para vincular y cargar fácilmente los ensambles.

Ensamblés para versionamiento e implementación

Los componentes .NET que son creados se ubican en ensambles, albergados dentro de EXEs o DLLs. Puede existir uno o más archivos físicos formando un ensamble. Un ensamble es un empaquetado de tipos lógicamente relacionados, y representa la unidad de versionamiento de software e implementación. Al desacoplar las nociones lógicas y físicas de un componente reutilizable, puede dividir su código en múltiples archivos como una opción de implementación. Por ejemplo, puede beneficiarse de una descarga incremental o bajo demanda, donde únicamente son descargados los tipos que son necesarios para la ejecución del programa. Al mismo tiempo, el conjunto relacionado de tipos sigue manteniéndose como una sola unidad de versionamiento e implementación dentro del ensamble. Los ensambles de múltiples archivos consisten de un conjunto de módulos. Un módulo es un archivo ejecutable portátil (PE) (un DLL o EXE), que consiste de uno o más tipos.

Los ensambles le permiten expresar las dependencias de versión estrictas entre las piezas de una aplicación y, debido a que son autodescriptivas, ayudan a habilitar la implementación xcopy.

Los ensambles también juegan un rol en el sistema de seguridad .NET en que el ensamble es la unidad en la que se solicitan y otorgan los permisos. Para reforzar adecuadamente el versionamiento, implementación y características de seguridad en .NET, el ensamble actúa como el rango de resolución que se utiliza para resolver referencias a los tipos.

Desde el punto de vista del consumidor del ensamble (la visión externa), el ensamble es una colección con nombre y versión de tipos exportados y recursos. Desde el punto de vista de un desarrollador de ensamble (la visión interna), un ensamble es una colección de uno o más archivos que implementan tipos y recursos.

La Metadata posibilita la mayoría de las características de la plataforma

Metadata es el elemento de unión que habilita muchas de las características del entorno Microsoft .NET, y es una descripción extremadamente completa de los elementos contenidos en el ensamble. Contiene detalles tales como descripciones de tipos e información acerca de los ensambles externos que un ensamble utiliza. Asimismo, la metadata contiene la información de versión, describe los recursos (por ejemplo imágenes) que se encuentran en el ensamble y permite otras características de .NET.

El CLR usa la metadata para ubicar y cargar los tipos de clase en el archivo, disposición de las instancias de objetos en memoria, resolver llamadas a métodos y referencias a campos, traducir MSIL a código nativo, forzar la seguridad y realizar muchas otras características adicionales.

La metadata de una clase la describe completamente, incluyendo sus métodos, parámetros de los métodos, convenciones de llamada, propiedades, los eventos que cada clase puede ejecutar, y la visibilidad de todos los miembros de la clase. La metadata está diseñada para ser la unión de todos esos atributos expuestos por cualquier lenguaje.

La Metadata proporciona la integración de múltiples lenguajes. Uno de los atractivos más importantes de .NET es la muy mejorada disponibilidad de fácilmente utilizar código escrito en diferentes lenguajes. Con el fin de hacerlo, los lenguajes que soportan .NET necesitan compartir un formato común para exponer información que describe un tipo. La metadata es ese formato común.

Otro beneficio de utilizar metadata es que los lenguajes .NET pueden usar mecanismos específicos de cada lenguaje para importar conocimiento acerca de componentes externos. Diferentes lenguajes utilizan diferente sintaxis para importar metadata, pero todos terminarán disponiendo de la misma información.

La Metadata permite la interoperabilidad

Metadata también es crucial para que .NET interopere con las APIs Win32 y objetos COM actuales. Cuando el código administrado llama una API no administrada, utiliza el mecanismo *Platform Invoke* (PInvoke). CLR .NET necesita realizar una preparación especial para llamadas PInvoke. Esto con frecuencia incluye *marshaling* de parámetros, por ejemplo la realización de conversión de tipos de datos para asegurar que el código no administrado recibe los tipos de parámetros que espera. Cada función y método invocable por código administrado debe tener metadata.

Además de proporcionar una manera consistente de importar datos acerca de componentes externos, metadata también crea una forma de describir los ensamblados en los que confía un componente determinado. Los componentes .NET no son referenciados en la registry, sino dentro de metadata. Debido a esto, simplemente tiene que copiar los archivos a un directorio para instalar la mayoría de las aplicaciones basadas en .NET. Para desinstalar la aplicación, simplemente elimine los archivos.

Los compiladores generan metadata

El trabajo de los compiladores es procesar su código fuente y producir el MSIL correspondiente. Además el compilador es responsable de embeber metadata en cada ensamblado.

Metadata es un super-conjunto de tecnologías anteriores tales como Bibliotecas de Tipos y Lenguaje de Definición de Interfaces (IDL). El aspecto importante que hay que resaltar es que metadata es mucho más completo que sus antecesores y siempre está relacionado con el archivo que contiene el código. Siempre está

incrustado en el mismo ensamble con el código, haciendo imposible separarlos.

Se requieren compiladores compatibles con .NET para emitir la información metadata completa acerca de cada clase de tipos en el módulo de código fuente compilado.

Las herramientas como Visual Studio.NET toman información metadata de un archivo utilizando una tecnología denominada *Reflection*, la cual permite la tecnología IntelliSense™ dentro de su editor de código fuente.

Los Componentes de un ensamble son descriptos en Manifestos

Los componentes de un ensamble se describen en un *manifest*. Un *manifest* es un bloque de datos que enumera los archivos del ensamble, y controla qué tipos y recursos quedan expuestos hacia afuera del ensamble. El *Manifest* también rige cómo las referencias a éstos tipos y recursos se correlacionan con los archivos que contienen sus declaraciones e implementaciones, y enumera otros ensambles de los que depende éste ensamble.

La existencia de un *manifest* proporciona una vía indirecta entre los consumidores del ensamble y los detalles de implementación del ensamble, y hace que los ensambles sean autodescriptivos.

Los ensambles determinan el alcance de los módulos

Cada módulo cargado en tiempo de ejecución lo hace en el contexto de un ensamble. A cada tipo que es cargado se le asigna el alcance del ensamble. Parte de la entidad de un tipo es su identidad de ensamble. Por ejemplo, un tipo denominado Cliente cargado en el alcance de un ensamble no es el mismo que el tipo Cliente cargado en el alcance de otro ensamble, incluso si el detalle de sus declaraciones e implementaciones son exactamente los mismos.

Como desarrollador, puede elegir opciones explícitas al momento del desarrollo. Si importa un tipo desde otro ensamble, la metadata de su archivo contiene referencias para ese ensamble. Si importa un tipo desde un módulo individual en un ensamble multiarchivos, su propio archivo contiene una dependencia explícita a ese módulo. En el último caso, .NET hace honor y fuerza los límites del ensamble en tiempo de ejecución, asegurándose de que su módulo y el referenciado formen parte del mismo ensamble.

El caché de ensambles mejora el desempeño

El caché de ensambles es un directorio del sistema que se utiliza para almacenar los ensambles que pueden ser utilizados por más de una aplicación. Cuando se instalan ensambles compartidos en una máquina, pueden colocarse en el caché de ensambles. El caché de ensambles consiste de dos caches lógicamente separados: el caché de ensambles global (GAC) y el caché de ensambles transitorio.

Cuando se distribuye un ensamble, se puede agregar a GAC. Cuando los ensamblados son descargados utilizando el explorador, el ensamblado se instala en el *caché* de ensamblados transitorio. Mantener los ensamblados lógicamente separados evita que un módulo descargado afecte seriamente una aplicación instalada.

BENEFICIOS PARA EL PROGRAMADOR

Ya que el enfoque de Microsoft .NET es proporcionar al desarrollador la plataforma contra la que desarrollar, en lugar de desarrollar directamente contra un sistema operativo, las ventajas más visibles del enfoque .NET son para los desarrolladores.

Programar Internet con servicios Web XML

Cuando desarrolla aplicaciones distribuidas basadas en Web con .NET, usted las crea utilizando los servicios Web XML, los cuales le ayudan a dirigir los retos de la integración de aplicaciones. Por ejemplo, le ayudan a tomar diferentes aplicaciones ejecutándose en diferentes sistemas operativos construidos con diferentes modelos de objetos utilizando diferentes lenguajes de programación y tornarlos en aplicaciones Web fáciles de utilizar.

Al igual que los componentes, los servicios Web representan funcionalidad caja-negra que puede reutilizarse sin preocuparse acerca de cómo el servicio es implementado. Puede ensamblar aplicaciones utilizando una combinación de servicios remotos, servicios locales y código personalizado.

Por ejemplo, podría ensamblar un almacén en línea utilizando el servicio Microsoft Passport para autenticar a los usuarios, un servicio de personalización de terceros para adaptar las páginas Web para cada usuario, un servicio de procesamiento de tarjeta de crédito, un servicio de impuesto por ventas, servicios de rastreo de paquetes de cada compañía de envíos, un servicio interno de catálogo que se conecte a las aplicaciones de administración de inventarios internos de la compañía, y un elemento de código personalizado que asegure que su tienda sobresale de entre la muchedumbre.

A diferencia de las actuales tecnologías de componentes, los servicios Web no utilizan protocolos específicos de modelos de objetos. Al contrario, se comunican utilizando protocolos Web existentes y formatos de datos tales como HTTP y XML. Cualquier sistema que soporte estos estándares Web puede soportar los servicios Web XML.

Además, un contrato de servicio Web, que forma parte del servicio Web, describe los servicios proporcionados en términos de mensajes que el servicio Web acepta y genera. Al enfocarse únicamente en los mensajes, el modelo de servicios Web es totalmente independiente del modelo de objetos, lenguaje y plataforma.

Acceso a datos para servicios Web

Casi todos los servicios Web necesitan consultar o actualizar los datos persistidos ya sea en archivos simples, bases de datos relacionales o en cualquier otro tipo de almacén de datos. Para proporcionar el acceso a datos, los servicios del entorno incluyen una biblioteca de clase ADO.NET. Como el nombre lo indica, ADO.NET ha evolucionado a partir de los Objetos de Datos ActiveX™ (ADO).

ADO.NET está diseñado para proporcionar servicios de acceso a datos para aplicaciones y servicios escalables basados en Web. ADO.NET proporciona APIs de flujo de datos de alto desempeño para el acceso conectado estilo de cursor a datos, así como un modelo desconectado a datos más adecuado para retornar

datos a las aplicaciones del cliente. Para la arquitectura ADO.NET, cualquier dato (sin importar cómo están actualmente almacenados) puede manipularse como XML o datos relacionales; lo que sea más importante para la aplicación en un momento determinado.

ADO.NET define clases para conectarse, emitir comandos y recuperar resultados de almacenes de datos. Los proveedores administrados de datos implementan estas clases. Los objetos *connection* y *command* de ADO.NET son similares a sus equivalentes en ADO, y una nueva clase denominada *DataReader* provee la capacidad de recuperar los resultados a través de una API de flujo de datos de alto desempeño. *DataReader* es equivalente funcionalmente a un *Recordset* de ADO de solo avance, sólo lectura; pero los *DataReaders* están diseñados para disminuir el número de objetos en memoria creados para evitar la recolección de basura y mejorar el desempeño. El entorno .NET incluye proveedores administrados de datos para Microsoft SQL Server y cualquier almacén de datos que puede accederse a través de OLE DB.

Una de las principales innovaciones de ADO.NET es la introducción de *Dataset*. *Dataset* es un caché de datos en memoria proporcionando una visión relacional de los datos. Consiste de una o más tablas junto con cualquier relación entre tablas y restricciones asociada. Los *Datasets* no saben nada acerca de la fuente de sus datos. Los *Dataset* se pueden crear y poblar programáticamente o bien cargando datos desde e almacén de datos. Sin importar de donde provengan los datos, se manipula utilizando el mismo modelo de programación y utiliza la misma memoria caché. La Figura 9 muestra la arquitectura básica de ADO.NET.

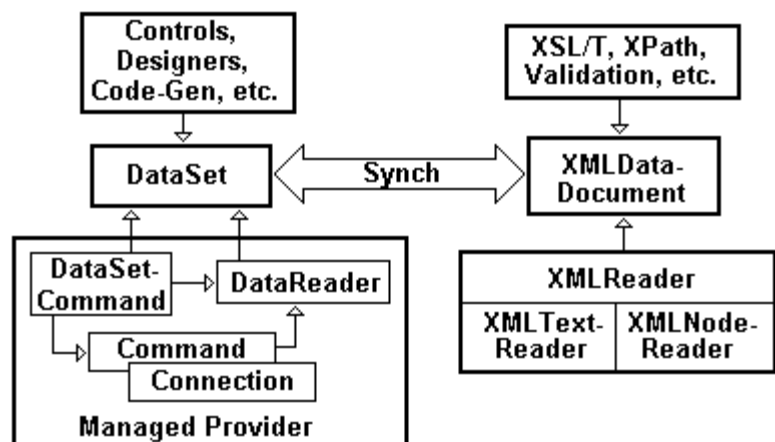


Fig. 9: Arquitectura ADO.NET

Desarrollo avanzado de aplicaciones Web con ASP .NET

ASP.NET es la más reciente evolución de la tecnología Active Server Page (ASP) actual. Sin embargo, ASP.NET ha sido creada desde cero, de manera que Ud. se beneficie de nuevas o grandemente mejoradas características, incluyendo:

- Lenguajes compilados orientados a objetos

- Separación más limpia entre código y contenido
- Mayor desempeño y escalabilidad
- WebForms para rápido desarrollo de páginas Web, incluyendo un modelo de eventos y controles Web
- Varios niveles de soporte de memoria caché
- Controles de servidor que pueden generar diferentes salidas, tales como HTML 3.2, dependiendo del cliente. Clientes ricos, incluyendo Internet Explorer, pueden explotar más funcionalidad.
- Desarrollo y acceso a servicios Web simplificado
- Acceso a una plataforma rica a través del entorno .NET

Las páginas ASP.NET tienen una variedad de nuevas extensiones. Una página ASP.NET básica utiliza .aspx como la extensión del nombre de archivo. Un Servicio Web utiliza .asmx, y una nueva construcción denominada controles de usuario utiliza .ascx. Estas nuevas extensiones de nombre de archivo existen por una razón. Las aplicaciones ASP.NET se ejecutan de lado a lado con las aplicaciones ASP existentes. No comparten el estado de sesión o el estado de la aplicación, y coexisten tranquilamente en el mismo servidor. Estas nuevas extensiones del nombre de archivo son necesarias de manera que Internet Information Services (IIS) pueda llamar al filtro ISAPI adecuado para procesarlos.

Otras nuevas características incluyen la adición de comentarios del lado del servidor (<%-- --%>) que le permiten comentar todo un bloque de código, expresiones de vinculación de datos (<%# %>) que le permiten unir los datos a controles de servidor, y nuevas directivas (<%@ %>) que le permiten establecer declarativamente un número de opciones de página.

ASP.NET aprovecha el CLR y el entorno de servicios para proporcionar un ambiente hosting confiable, robusto y escalable para las aplicaciones Web. Asimismo, ASP.NET se beneficia del modelo de ensambles de CLR para simplificar la implementación de aplicaciones. Además, proporciona servicios para simplificar el desarrollo de aplicaciones (como los servicios de administración de estado) y modelos de programación de más alto nivel (tales como Web Forms y servicios Web de ASP.NET). La Figura 10 muestra la arquitectura del sistema ASP.NET.

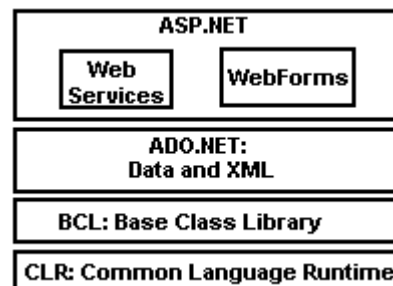


Fig. 10: Arquitectura ASP.NET

Código más limpio

ASP.NET proporciona la característica de “código detrás” que le permite separar el contenido específico de presentación de la lógica de la aplicación. El código contenido dentro de los métodos de evento puede residir en un archivo separado del archivo .aspx el cual contiene el código de presentación (por lo regular HTML). Su lógica de la aplicación puede desarrollarse utilizando cualquiera de los lenguajes .NET que se ejecutan dentro de CLR, lo que significa que todas las ventajas asociadas están disponibles para el código ASP.NET (por ejemplo, *debugging*, compilación, herencia seguridad colección de basura, escalabilidad, facilidad de implementación, etc.). Separar el código en archivos separados hace que este sea mucho más fácil de leer. Otra de las ventajas es que permite a su departamento de diseño dar una apariencia profesional al sitio al tiempo que los programadores se encargan de la lógica de negocios.

Existen otras ventajas por las que dividir el contenido del estilo. Una es que se pueden utilizar herramientas de diseño estándar para construir la interfaz del usuario sin agregar complejidad al código. Otra es que la localización se vuelve mucho más simple ya que la página de interfaz del usuario consiste únicamente de HTML, en lugar de una mezcla de HTML y código.

Todo el código ASP.NET se compila y ya no hay ninguna ejecución interpretada. Aunque las páginas ASP.NET se compilan por completo, puede actualizar sus páginas sin tener que reiniciar su proceso IIS. Esto le permite mantener el estilo ASP simple de creación e implementación al tiempo que aprovecha todo el código compilado.

El movimiento a lenguajes definidos fuertemente en base a tipos también significa que ya no va a estar restringido al uso de tipos de datos *variant*. Ahora puede declarar variables y objetos de tipos sólidos, y unirse tempranamente a esos objetos.

Verá que el desempeño aumenta significativamente cuando migra código ASP si cambia todas sus instancias de variables de tipos libres a tipos de datos definidos. Este cambio por si solo puede cosechar importantes aumentos en el desempeño. Adicionalmente, al reescribir los componentes COM existentes como código administrado se eliminarán muchas de las sobrecargas de desempeño relacionadas con *marshaling*, *threading* e interoperabilidad COM.

Asimismo, ASP.NET introduce diversos cambios al significado básico de la página. Desde el punto de vista de compatibilidad y migración, los más importantes se mencionan a continuación.

- Usted puede tener únicamente un lenguaje por página. No obstante, puede seguir teniendo JavaScript del lado del cliente y Visual Basic.NET del lado del servidor en la misma página. Observe que los controles de usuario (una nueva característica que le permite crear elementos reutilizables que se pueden incrustar en otras páginas) pueden escribirse en un lenguaje diferente al utilizado en la página.

- Las funciones deben estar bloques de scripting (si está utilizando código embebido en lugar del archivo de código detrás). Por ejemplo, en lugar de escribir:

```
<%
    Function MyFunc()
    .....
    End Function
%>
```

Deberá escribir:

```
<SCRIPT LANGUAGE="VB" runat="server">
    Function MyFunc()
    .....
    End Function
</SCRIPT>
```

- Las funciones *Render* no son soportadas. Como efecto colateral de la manera en que el scripting fue implementado en ASP, era posible escribir funciones como esta:

```
<% Function DrawTable() %>
    <table>
    <tr>
    <td>
    <%
        Response.Write "Hello World" %>
    </td>
    </tr>
    </table>
<% End Function %>
```

Con ASP.NET, debido a que las funciones deben ser colocadas en bloques de scripting, donde no hay una forma fácil de incluir HTML incrustado, el código anterior se debe cambiar para que se vea como:

```
<SCRIPT language="VB" runat="server">
    Function DrawTable()
        Response.Write "<table><tr><td>"
        Response.Write "Hello World"
        Response.Write "</td></tr></table>"
    End Function
</SCRIPT>
```

Compatibilidad del explorador

El uso que ASP.NET hace de los Web Forms integrados con los controles de servidor que generan automáticamente HTML, proporciona la ventaja adicional de compatibilidad automática con los exploradores. Con el modelo de programación de Web Forms, usted diseña su interfaz de usuario utilizando controles de servidor que se ejecutan en el servidor. Estos controles son capaces de detectar el tipo de explorador en tiempo de ejecución, y les envían de regreso HTML 3.2 o Dynamic HTML según sea necesario. El punto importante es que Ud. expresa su lógica únicamente una vez usando eventos del lado del servidor.

En Visual Basic usted dibuja los controles en un formulario, y después implementa los procedimientos de eventos. ASP.NET le proporciona el mismo modelo de programación. En lugar de tener que obtener manualmente los valores de variables

de diferentes formularios, puede escribir el siguiente código:

```
<html>
<script language="vb" runat="server">
    Sub SubmitButton_Click(Source As Object, E As EventArgs)
        YouEntered.Text = "You entered " & txtName.text
    End Sub
</script>
<body>
<form method="post" runat="server">
    Name: <asp:textbox id="Name" runat="server" /><br>
    <asp:Button text="Enter" OnClick="SubmitButton_Click" runat="server"
/>
    <br>
    <asp:label id="YouEntered" runat="server" />
</form>
</body>
</html>
```

Observe el uso del atributo `runat="server"`, y el uso de los elementos precedidos con `asp:` namespace. Los elementos `asp:` identifican los controles Web, y el atributo `runat="server"` indica que son controles del lado del servidor, procesados en el servidor.

Asimismo observe el evento `OnClick`. Este evento se activa en el servidor debido a que el atributo `runat="server"` se estableció para el control correspondiente. Observe como el atributo `OnClick` relacionado con el control `asp:Button` se utiliza para cablear el evento al controlador de evento, en este caso `SubmitButton_Click`.

Asimismo, observe cómo puede referenciar controles en el código que se ejecuta en el servidor. Por ejemplo, el código anterior en el controlador de eventos, establece la propiedad de texto `asp:label` utilizando la ID del control.

El código se comporta como una aplicación tradicional Visual Basic basada en formularios, y es mucho más intuitivo de codificar que la lógica ASP equivalente. Puede utilizar los controles del servidor para vincular procedimientos de eventos con código basado en el servidor. Estos controles basados en servidor envían HTML puro a los exploradores de nivel bajo, y no hay scripting involucrado del lado del cliente.

Aunque la salida predeterminada de los controles Web de ASP.NET es HTML 3.2, también pueden emitir DHTML para exploradores adecuados. Este le permite crear páginas utilizando un solo conjunto de controles de servidor, permitiendo que los controles por sí mismos decidan qué tipo de salida enviarán dependiendo del explorador. Esto le permite tener controles que envíen DHTML y scripting del lado del cliente para versiones recientes de Internet Explorer, y HTML 3.2 para otros exploradores.

Modelo Sin Estado

El Web es un modelo fundamentalmente sin mantener estado, sin correlación entre solicitudes HTTP. Esto hace difícil la escritura de aplicaciones Web, ya que las aplicaciones por lo regular necesitan mantener su estado a través de las múltiples solicitudes.

ASP.NET mejora los servicios de administración de estado presentados por ASP

para proporcionar tres tipos de estado para las aplicaciones Web: aplicación, sesión y estado de Vista.

- Estado de aplicación. Al igual que ASP, el estado de aplicación es específico para una instancia de aplicación y no se persiste.
- Estado de sesión. Esta es específica para una sesión de usuario con la aplicación. A diferencia del estado de sesión ASP, el estado de sesión ASP.NET se puede almacenar en un proceso separado, y hasta puede configurarse para ser almacenado en una máquina separada. Esto hace que el estado de sesión sea utilizable cuando se implementa la aplicación en un Web Farm.
- Estado de Vista. Esto se refiere al estado de una página y sus propiedades actuales, junto con el estado de cualquiera de los controles de la página. Se asemeja al estado de sesión pero no tiene vencimiento y se guarda automáticamente con cada ronda del cliente al servidor. El estado de Vista también es útil para almacenar preferencias del usuario y otra información de personalización.

Como un ejemplo del estado de vista, el código anterior genera el siguiente HTML 3.2:

```
<html>
<body>
<FORM name="HtmlForm2" method="post" action="Test.aspx" id="HtmlForm2">
  <INPUT type="hidden" name="__VIEWSTATE" value="a0z664351470__x">
  Name: <input name="txtName" type="text" id="txtName">
  <br>
  <input type="submit" name="Button5" value="Enter">
  <br>
  <span id="lblYouEntered"></span>
</FORM>
</body>
</html>
```

Lleva a cabo un post estándar de formulario que manda la entrada del usuario nuevamente al mismo archivo. No hay un estado de servidor mantenido y no hay scripting del lado del cliente que mantenga el estado. El campo __VIEWSTATE oculto mantiene el estado del control. Esto significa que los controles retienen automáticamente su estado entre los *post-backs* de la página sin ninguna intervención de programación.

ASP.NET soporta movilización de sesiones no solo fuera del proceso sino fuera de la máquina. Esto significa que puede crear un servidor de estado dedicado que controle el estado para múltiples servidores en un Web farm. Aunque existe un hit de desempeño para mover el estado de sesión fuera del proceso, y mucho mayor cuando se mueve fuera de la máquina, la ventaja es enorme. El estado de sesión ahora puede sobrevivir a reinicios de la aplicación así como reinicios de la máquina. También puede operar a través de *firewalls* y conexiones enrutadas.

Un segundo proveedor de estado de sesión también ha sido implementado, el cual utiliza SQL Server como un almacén duradero de datos. Esto permite que los datos de la sesión sobrevivan al reinicio del servidor de sesión, aunque esto es con mayor costo para el desempeño.

Una de las limitantes principales del estado de sesión ha sido su dependencia en *cookies* para reconectar a los usuarios a su almacén de estado en el servidor. ASP.NET ahora ofrece un modelo de estados de *cookie*, el cual trabaja incluyendo la información normalmente almacenada en la *cookie* en URLs.

Web Forms

Los Web Forms traen los beneficios de productividad del desarrollo de formularios Visual Basic al desarrollo de aplicaciones Web. Las Web Forms permiten a los desarrolladores crear rápidamente aplicaciones Web programables inter-plataforma, inter-explorador, utilizando casi las mismas técnicas que se utilizaron anteriormente para crear aplicaciones de escritorio basadas en formularios.

Los Web Forms soportan la sintaxis ASP tradicional que mezcla el contenido HTML con código scripting, y también promueve un enfoque más estructurado que separa el código de la aplicación del contenido de interfaz de usuario. Una página Web Forms estándar consiste de un archivo HTML que contiene la representación visual de la página, y un archivo con el código fuente de manejo de eventos. El fuente se compila en código ejecutable, proporcionando rápido desempeño en tiempo de ejecución. Ambos archivos residen y se ejecutan en el servidor donde generan un documento HTML 3.2 que es enviado al cliente.

El construir un formulario Windows en Visual Basic requiere agregar un formulario al proyecto, arrastrar un control al formulario, y después hacer doble clic en el control para escribir código. Los Web Forms proporcionan los mismos principios de diseño (y los niveles de productividad) al Web.

Los controles Web Form son responsables por un modelo de programación basado en componentes controlados por eventos, típicamente generando la interfaz de usuario en forma de HTML. ASP.NET viene con un conjunto de controles Web Form que reflejan los elementos HTML típicos de interfaz de usuario (incluyendo listboxes, cajas de texto y botones), y un conjunto de controles Web que son más complejos (tales como calendarios y ad-rotators).

Web Forms y los controles Web Form ocultan las complejidades de la administración de estado a través de la interacción del usuario con una página. También se proporcionan ricos servicios de data-binding, facilitado a los desarrolladores de aplicaciones el mostrar los datos recuperados a través de servicios de acceso de datos.

La separación de el código y el contenido permite que las páginas ASP.NET se compilen dinámicamente en clases administradas para lograr rápido desempeño. Cada solicitud HTTP entrante es entregada a una nueva instancia de la página, de manera tal que los desarrolladores no tengan que preocuparse acerca de la seguridad de hilos de ejecución en su código.

Desde el punto de vista del diseño, Web Forms:

- Utiliza un modelo de programación que es instantáneamente familiar para la mayoría de los desarrolladores. Los desarrolladores pueden moverse

fácilmente entre proyectos de escritorio y de Web sin necesidad de tener que reentrenarse extensivamente.

- Separa la disposición HTML del código detrás de la página. Esta separación facilita aún más actualizar cualquiera de las partes por separado, simplifica el código y permite que el código sea versionado más fácilmente.
- Se puede crear con cualquier herramienta soportada en Visual Studio.NET. Las páginas HTML también se pueden importar desde herramientas como FrontPage y convertirse en Web Forms.

También existen ventajas en tiempo de ejecución para Web Forms:

- Proporcionan un modelo de programación y un entorno de ejecución que permite a los desarrolladores implementar páginas HTML generadas en el servidor.
- Mejoran grandemente el desempeño en tiempo de ejecución debido a que el código detrás de la página HTML es compilado, no interpretado.
- Generan páginas HTML 3.2 cuando se visualizan con exploradores de bajo nivel. Como resultado, los exploradores de cualquier plataforma pueden ver las páginas.

Controles del servidor

ASP.NET encapsula gran cantidad de funcionalidad en común a través de los controles del servidor. Por ejemplo, el control ad-rotator ha sido actualizado y ahora utiliza XML para almacenar su información. También se proporcionan controles para administrar el estado de un formulario, y mostrar calendarios y tablas. Para casi todos los elementos HTML existe un control ASP.NET del servidor que lo produce y le permite interactuar con él programáticamente. Es decir, ya no tendrá que codificar un bucle con una condición para mantener una opción seleccionada en un cuadro de lista. Simplemente puede decirle al cuadro de lista que se ejecute en el servidor y que lo administre por usted. Asimismo, puede decirle programáticamente al cuadro de lista el elemento a mostrar como seleccionado.

Es posible que uno de los nuevos controles más interesantes sea DataGrid. Este es una grilla conectada a datos, de múltiples columnas, que se puede vincular fácilmente a un dataset ASP.NET. Soporta paginación y ordenamiento, y se pueden editar filas individualmente, todo esto con muy poco código.

Los controles de servidor son nuevos, pero no son difíciles de aprender debido a que se correlacionan con sus equivalentes HTML. Los controles de servidor son identificados por el espacio de nombre 'asp:', y utilizan el método XML de una barra al final para cerrar el elemento. Al tiempo que no tiene la restricción de utilizar este estilo XML, y pueda utilizar el estilo HTML con una etiqueta de cerrado separada (</asp:TextBox>), éste es más rápido de escribir. El espacio de nombre tiene que ser utilizado debido a que identifica de dónde proviene el control. Cada uno de los controles Web estándar forman parte del espacio de nombre ASP. Esto se vuelve importante cuando comienza a crear sus propios controles.

El control TextBox puede parecer que no ofrezca una ventaja sobre el cuadro Input estándar, pero deberá considerar los siguientes tres controles Input:

```
<input type="text" ...>
<input type="password" ...>
<textarea rows="5" ...>
```

Todos son utilizados para entrada en HTML, pero no hay consistencia. Considere qué tan simple es lo siguiente:

```
<asp:TextBox runat="server" ...>
<asp:TextBox mode="Password" ...>
<asp:TextBox Rows="5" ...>
```

Un simple control puede proporcionar la funcionalidad de tres, dependiendo de cómo se utilice. De hecho, estos controles TextBox generan la misma salida HTML. Sin embargo, es mucho más fácil recordar y codificar.

ASP.NET se entrega con cinco familias de controles de servidor:

- Controles intrínsecos que correlacionan sus equivalentes HTML,
- Controles de lista para proporcionar flujo de datos a través de una página,
- Controles ricos para proporcionar interfaz de usuario más rica de contenido y funcionalidad,
- Controles de validación para realizar una variedad de revalidaciones en formularios,
- Controles móviles para encapsular WML para dispositivos WAP.

Los controles de servidor intrínsecos son similares a los controles HTML, pero han sido racionalizados para proporcionar un uso más consistente. Estos controles incluyen LinkButton, ImageButton, HyperLink, TextBox, CheckBox, RadioButton, DropDownList, ListBox, Image, Label, Panel, Table, TableRow y TableCell.

Los controles de lista incluyen Repeater, DataList y DataGrid. Los controles de lista también incluyen un RadioButtonList y CheckBoxList para fácil creación de listas de botones de radio y cuadros de chequeo.

Los controles ricos incluyen Calendar y AdRotator. El control Calendar generará HTML puro para exploradores de bajo nivel, o DHTML para exploradores de nivel más alto. AdRotator emite imágenes, y tiene código de rotación integrado.

Los controles de validación incluyen RequiredFieldValidator, CompareValidator, RangeValidator, RegularExpressionValidator, CustomValidator y ValidationSummary. Estos proporcionan maneras fáciles para que los desarrolladores creen validación en el manejo de formularios.

Es importante darse cuenta que los controles del lado del servidor no son controles COM o controles de tiempo de diseño. Son construcciones que existen únicamente en el servidor y se otorgan como HTML estándar para el cliente.

Modelo orientado a objetos

ASP.NET ha sido reescrito con orientación a objetos en mente. ASP.NET proporciona un modelo de objetos para la página. En la parte superior de la

jerarquía de objetos se encuentra el objeto Page. Cada control, aplicación y página en ASP.NET hereda de Page. Esto significa que cada página es una instancia del objeto Page. El evento Load de la página es uno de los eventos más importantes a los que puede responder:

```
<html>
<script language="VB" runat="server">
    Sub Page_Load(Source As Object, E As EventArgs)
        ' code to run when page loads
    End Sub

    Sub SubmitButton_Click(Source As Object, E As EventArgs)
        ' code to run when button is clicked
    End Sub

    Sub Page_Unload(Source As Object, E As EventArgs)
        ' code to run when page unloads
    End Sub
</script>

<form runat="server">
    <asp:Button text="Enter" OnClick="SubmitButton_Click" runat="server" />
    <asp:label id="YouEntered"/>
</form>
</html>
```

Aquí, el evento Load se lanza a medida que la página se carga, cuando todos los controles basados en servidor están disponibles para su uso. Durante la interacción con el usuario, se generan otros eventos, y después se activa el evento Unload cuando la página se descarga.

Adicionalmente, cada objeto en una página puede tener su propio modelo de eventos y exponer y ejecutar eventos de servidor manejados por sus métodos de evento del lado del servidor. Rutinas como Button_Click or Listbox_Change pueden hacer relativamente simples el procesamiento estándar de formularios y varias otras tareas comunes. La legibilidad del código también se mejora ampliamente.

Controles de usuario

Usted no está limitado a utilizar los controles proporcionados, y es muy sencillo crear los suyos propios. Por ejemplo, si desea un simple control para encapsular dos cuadros de texto, por ejemplo para los campos de nombre y apellido, podría escribir código que luzca como esto:

```
<asp:Panel runat="server">
    <asp:Textbox id="txtFirstName" text="First Name"
        runat="server" />
    <asp:Textbox id="txtLastName" text="Last Name"
        runat="server" />
</asp:Panel>
```

puede guardar este código en un archivo nombrado Name.aspx (observe la nueva extensión), y tratarlo como si fuera un control Web Form. Por ejemplo, puede agregar lo siguiente a su Web Form:

```
<%@ Register TagName="NameControl" TagPrefix="Foo" Src="Name.aspx" %>
<form>
  <foo:NameControl runat="Server"/>
</form>
```

Puede crear código fácilmente reutilizable. También puede crear código directamente en cualquier lenguaje .NET, subclasificarlos desde otros controles, y hacer que se emitan en cualquier salida que se requiera. Debido a que los controles se califican utilizando espacios de nombre, no deberán haber colisiones de nombres de control. Esto crea un ambiente de programación muy extensible. Es muy probable que habrá un gran mercado de terceros para ricos controles.

Otra forma de separar el código de la página, es colocar el código en una biblioteca y después cambiar el bloque de script para hacer referencia a la biblioteca, como se muestra a continuación:

```
<SCRIPT runat=server SRC="mycode.dll">
```

Esta línea de código conecta automáticamente todos los eventos de manera dinámica al DLL compilado.

Controles para Conexión a Datos

Uno de los nuevos controles Web es el DataGrid, el cual tiene soporte integrado para mostrar conjuntos de datos. Para generar una tabla HTML de datos generados desde SQL, simplemente cree objetos ADO.NET y ejecute el comando necesario para obtener los datos, el que usa como la fuente para el DataGrid.

```
<%@ Import Namespace="System.Data.SQL" %>
<html>
<script language="VB" runat="server">
  Sub Page_Load(Sender As Object, E As EventArgs)
    Dim myCommand As SqlCommand
    myCommand = New SqlCommand("select * from products",
                               "Server=localhost; Database=AdvWorks; UID=sa")
    DataGrid1.DataSource = myCommand.Execute
    DataGrid1.DataBind
  End Sub
</script>

<body>
  <ASP:DataGrid id="DataGrid1" runat="server" />
</body>
</html>
```

Todo lo que tiene que hacer es vincular los datos al data grid, y después se produce la tabla HTML. La vinculación a datos no es sólo restringido para datos de bases de datos. También puede vincular a tablas hash, arreglos, otros controles de servidor, propiedades de una página, y XML.

Si utilizar el conjunto predeterminado de columnas no se ajusta a sus necesidades, puede personalizarlo para mostrar únicamente aquellas en los que esté interesado.

```

<asp:DataGrid id="DataGrid1" AutoGenerateColumns="false" runat="server">
  <property name="Columns">
    <asp:BoundColumn HeaderText="Name" DataField="ProductName"/>
    <asp:BoundColumn HeaderText="Description"
DataField="ProductDescription"/>
  </property>
</asp:DataGrid>

```

Los controles Repeater y DataList también emplean plantillas, que le permiten personalizar la apariencia del control. Ni Repeater ni DataList tienen interfaces de usuario predeterminadas, por lo que las debe proporcionar utilizando plantillas. Las plantillas se codifican en una forma similar para ambos tipos de control.:

```

<asp:DataList id="DataList1" runat="server">
  <template name="HeaderTemplate">
    Here's your list of titles<br>
  </template>
  <template name="ItemTemplate">
    <%=# DataBinder.Eval(Container.DataItem, "Title") %> <br>
  </template>
</asp:DataList>

```

Con este uso de plantillas puede especificar qué controles HTML conformarán cada parte del control vinculado a datos. Existen cinco nombres de plantillas que pueden utilizar con el control DataList: HeaderTemplate para la parte superior de su control, ItemTemplate para cada elemento, Alternating-ItemTemplate para cada otro elemento, SeparatorTemplate para el área entre cada elemento, y FooterTemplate para la parte inferior de su control.

La ventaja de este enfoque es que tiene gran cantidad de control sobre lo que exactamente desea que muestre su interfaz:

```

<asp:DataList id="MyDataList" RepeatColumns="2" runat="server">
  <template name="itemtemplate">
    <table cellpadding=10 style="font: 10pt verdana">
      <tr>
        <td width=1 bgcolor="BD8672"/>
        <td valign="top">
          <img align="top" src='<%=# DataBinder.Eval(Container.DataItem,
"ProductImageURL") %>' >
        </td>
        <td valign="top">
          <b>Name: </b>
          <%=# DataBinder.Eval(Container.DataItem, "ProductName")
%><br>
          <b>Description: </b>
          <%=# DataBinder.Eval(Container.DataItem,
"ProductDescription") %><br>
          <b>Price: </b>
          <%=# DataBinder.Eval(Container.DataItem, "ProductPrice", "$
{0}") %>
        </td>
      </tr>
    </table>
  </template>
</asp:DataList>

```

Este código es relativamente simple y no requiere mucho más código que el control DataList anteriormente mostrado. Usted puede especificar el número de columnas que aparecerán, y la lista automáticamente maneja el empaquetado de las columnas.

Controles de Validación

Los controles de validación ASP.NET:

- Proporcionan componentes que pueden realizar el 90% más de las tareas de validación de entrada típicas para las páginas de entrada de datos.
- Realizan validación basada en scripts de lado del cliente en exploradores modernos, y retroceden a validación HTML 3.2 pura basada en el servidor, si se requiere.
- Proporcionan una API flexible de manera que cualquiera de las tareas de validación no cubiertas por los componentes suministrados son fáciles de completar.

Un control de validación es un control visual ASP.NET que verifica una condición de validación específica de otro control. Por lo general aparece ante el usuario como una pieza de texto que se muestra u oculta dependiendo si el control que está verificando contiene un error. También puede ser una imagen, o inclusive puede ser invisible y seguir haciendo trabajo útil. Existen cinco tipos de controles de validación que realizan diferentes tipos de chequeos (evaluación de expresiones).

Otro control de validación clave es el control `ValidationSummary`. Las grandes páginas de entrada de datos por lo general tienen un área donde aparecen todos los errores. `ValidationSummary` genera automáticamente este contenido consolidado de resumen de errores obteniéndolo desde los controles individuales de validación en la página.

Por último, el objeto `Page` expone por sí mismo una propiedad `IsValid`, que verifica en código de servidor para determinar si es válida la entrada del usuario. Siempre debería validar en el servidor los datos de entrada, además de en el cliente, ya que es relativamente fácil para un usuario malicioso editar HTML y evitar la validación del lado del cliente.

Servicios Web XML

La infraestructura de servicios Web de ASP.NET proporciona diversos beneficios que simplifican el desarrollo y utilizan un modelo de programación que será muy familiar para los desarrolladores que trabajaron con ASP o Visual Basic. No necesitan entender los detalles de HTTP, SOAP, WSDL, o cualquier otra especificación de servicios Web, para usar este modelo de programación.

Los clientes pueden someter solicitudes a través de SOAP, HTTP GET, y HTTP POST. Se definen convenciones para métodos de encriptación y parámetros como cadenas de consulta para HTTP GET y datos de formulario para HTTP POST. HTTP GET y HTTP POST no son mecanismos lo suficientemente potentes como SOAP, ya que tipos de datos complejos no se pueden representar, pero permiten que los clientes que no soporten SOAP puedan acceder al servicio Web.

El modelo de servicios Web de ASP.NET asume una arquitectura de servicios sin mantener estado. Las arquitecturas sin mantener estado por lo general son más

escalables que las arquitecturas que mantienen estado. Los servicios Web pueden utilizar los servicios de administración de estado de ASP.NET si necesitan mantener el estado a través de las solicitudes. Los servicios Web basados en ASP.NET son aplicaciones administradas ejecutadas por CLR, y también se benefician de todas las características de CLR y .NET Framework analizados anteriormente.

ASP.NET también suministra una herramienta en el SDK (wsdl.exe) para generar proxies fuertemente administrados escritos para cualquier servicio Web descrito por un archivo de Lenguaje de Descripción de Servicios Web (WSDL). El generador proxy correlaciona los mensajes descritos en un archivo WSDL en métodos de las clases generadas. El proxy oculta una gran cantidad de infraestructura de red y traducción de variables del código de la aplicación, por lo que al utilizar el servicio Web solo se ve como si se estuviera utilizando otra clase administrada. El proxy preferirá utilizar SOAP para comunicarse con el servicio Web, pero también soporta los mecanismos HTTP GET y HTTP POST.

Para desarrollar un servicio Web, en lugar de escribir un archivo .aspx, usted escribe un archivo .asmx, el cual contiene una clase con diversos métodos que desea exponer. Cuando apunta un explorador a un archivo .asmx, éste se compila, y se genera el contrato WSDL requerido, y se retorna una página predeterminada que documenta el servicio, proporcionando una manera sencilla de probarlo.

Los servicios Web le permiten exponer sus funciones de negocios personalizadas (tales como detalles de rastreo de paquetes) para el público a través del Web. Para hacerlo, se escribe una clase que exponga sus métodos como URIs y que retorne datos como XML. Por ejemplo, una compañía podría llamar al servicio de rastreo de la compañía de envíos, e incorporar los resultados del rastreo en su propia aplicación de monitoreo de pedidos. Aquí está cómo la compañía de envíos podría crear un servicio Web utilizando C#:

```
<%@ WebService language="c#" Class="Shipping"%>
using System.Web.Services;
public class Shipping {
    [WebMethod]
    public string OrderStatus(string OrderNumber) {
        // code here to retrieve order details from data store
        return Status;
    }
}
```

este código se puede colocar en un archivo denominado Tracking.asmx en el directorio de aplicaciones en el sitio Web de la compañía de envíos. La compañía cliente puede llamar a este servicio Web en determinado número de formas. Podría utilizar HTTP-GET, donde los parámetros del método podrían pasarse junto con la cadena de consulta. Por ejemplo:

`http://orders.ups.com/orders/Tracking.asmx/OrderStatus?OrderNumber=BRU123`

La compañía cliente podría utilizar HTTP-POST, donde los parámetros del método se pasan dentro del cuerpo del post, como valores del formulario. O podría utilizar HTTP-SOAP, donde los parámetros del método se empaquetan en un formato XML estándar en la industria.

Ahora el cliente final puede simplemente consultar la compañía cliente (su proveedor) por sus detalles de pedido, y la aplicación Web de la compañía cliente obtendrá la información de rastreo de la compañía de envíos y la regresará junto con el estado del producto. La compañía cliente también podría exponer sus estados de pedidos como un servicio Web para permitir que otras personas lo utilicen.

Lo fabuloso acerca de los servicios Web es que le permiten exponer un servicio sin exponer datos o reglas de negocios particulares. Su código y datos se mantienen seguros al tiempo que suministra automáticamente sus servicios de negocios.

Memoria caché mejorada

Para desarrollar un sitio Web escalable de alto desempeño, inevitablemente se requiere alguna forma de memoria caché. Ciertas operaciones de construcción de páginas son costosas de construir, y guardarlas en memoria caché una vez que han sido construidas, ahorra recursos de servidores y les permiten ser servidos de manera más rápida. ASP.NET ofrece lo siguiente:

- Cacheo de página, permite que se guarden en memoria caché páginas enteras
- Memoria caché de fragmentos, que permite que se guarden en la memoria caché partes de páginas
- API caché, que permite a los programadores acceder al mecanismo de la memoria caché.

El cacheo de páginas es la forma más simple de guardar algo en una memoria caché, y permite que páginas dinámicas sean guardadas en memoria caché de salida, de manera que se sirvan directamente desde la memoria caché en lugar de que se ejecuten en cada solicitud. Puede especificar ya sea un tiempo absoluto (como media noche) o tiempo relativo (como 20 minutos después de que la página haya sido accedida por última vez) para un control fino de cuánto tiempo permanecen las páginas en la memoria caché. El cacheo de páginas es sensible al URL solicitado y a la cadena de consulta, de manera que la página a la que le pasan los parámetros únicamente regresará la versión guardada en la memoria caché si los parámetros coinciden. La memoria caché de fragmentos ofrece el mismo grado de control sobre partes individuales de una página ASP.NET, la cual permite subsecciones de una página en la memoria caché al tiempo que otras partes de la página permanecen dinámicas.

Estas dos técnicas utilizan la API de Cache, la cual se expone para que el programador le permita guardar en la memoria caché sus propios objetos, en lugar de recrearlos cada vez que sean necesarios. Esta API de cache incluye políticas de expiración así como notificaciones de cambios en archivos que le permiten hacer un uso inteligente de la memoria caché para aumentar significativamente el desempeño.

La memoria caché es especialmente valiosa para sitios tales como catálogos de productos donde el contenido necesita ser dinámico, pero no hay cambios frecuentes. Con ASP, si necesita un catálogo dinámico debe recrearlo desde la

base de datos con cada solicitud de página, o bien convertirla a HTML. Con el cacheo de página en ASP.NET puede especificar que la página dinámica sea cacheada en memoria durante un día; de manera que su salida es cacheada con el primer requerimiento a la página en el día. Los pedidos subsecuentes a la página se sirven desde la memoria caché.

Depuramiento

ASP.NET se basa en CLR, como resultado puede depurar sin problemas el código dentro de la página ASP.NET, luego en un objeto, y regresar nuevamente. Debido a la manera en que se compilan las páginas ASP.NET, el .NET Framework también puede analizarlas a medida que son ejecutadas, lo que proporciona un rico ambiente de depuración inter-lenguaje. Puede pasar desde su página Web basada en Visual Basic.NET a un control que haya sido escrito en C#.

Rastreo

ASP.NET introduce rastreo tanto en la página como en la aplicación. Con el soporte ASP.NET para rastreo de página, un rastreo de página junta todos los detalles de solicitudes de objetos incluyendo el árbol de control del servidor, variables del servidor, encabezados, cookies, y parámetros de cadenas de formularios/consultas, y los coloca en una tabla formateada en la parte inferior de cada página. También inserta una bitácora de todos los eventos ejecutados en la página, junto con tiempos exactos.

El rastreo esta construido en ASP.NET a través del método Trace de la página. Por ejemplo, para rastrear la entrada y salida a un procedimiento de evento, podría utilizar el siguiente código:

```
Sub Page_Load(Source As Object, E As EventArgs)
    Trace.Write "Page_Load", "Start"
    ' rest of code goes here
    Trace.Write "Page_Load", "End"
End Sub
```

Para permitir la salida de esta información de rastreo, usaría una directiva de Page como esta:

```
<%@ Page Trace="True" %>
```

Si desea información de rastreo pero no quiere que los usuarios la vean, puede configurar la aplicación para que envíe la información a otra ubicación donde pueda ser examinada posteriormente.

Implementación

ASP.NET utiliza el modelo de implementación basado en ensamblajes del Microsoft .NET Framework, y como resultado se beneficia de características tales como implementación xcopy, implementación de lado a lado de ensamblajes, y configuración basada en XML.

ASP.NET también soporta la implementación de una aplicación totalmente compilada. El beneficio es que ningún código fuente es visible para el administrador del servidor Web, una característica importante si otra compañía alberga su aplicación.

ASP.NET cuenta con un modelo de implementación extremadamente simple, y para implementar una aplicación, solo copie todos los archivos que componen la aplicación en el directorio adecuado. No se requiere registro de objetos o reinicio de aplicaciones. Todas las partes de una aplicación ASP.NET se pueden implementar en esta forma, incluyendo páginas, servicios Web, componentes compilados (contenidos dentro de DLLs), e incluso datos de configuración.

Para componentes compilados existe ahora un directorio “*bin*” especial bajo cada raíz de aplicación. Cualquiera de los componentes colocados en este directorio se ponen a disposición automáticamente para usarse dentro de esa aplicación. Ya que únicamente están disponibles para esa aplicación específica, se proporciona un aislamiento real de la aplicación. Cada aplicación en el servidor podría ejecutar una versión ligeramente diferente del componente sin efecto en otras aplicaciones. Esto es una gran ventaja para los desarrolladores que albergan múltiples instancias de un sitio en una máquina, como un ambiente de desarrollo, un ambiente de prueba, y un ambiente intermedio, cada uno con diferentes versiones de los componentes.

ASP.NET ya no bloquea DLLs para evitar actualizarlos. Simplemente puede copiar el nuevo DLL sobre el anterior. ASP.NET detecta automáticamente el cambio y migra a la nueva versión. Las solicitudes actualmente en ejecución siguen utilizando la versión anterior, mientras que las nuevas solicitudes se dirigen directamente a la nueva versión. Ya no tiene que rearmar o reiniciar la aplicación. Para desinstalar completamente la aplicación, o parte de ella, simplemente elimine los archivos relevantes. Ya no tendrá que desregistrar componentes y eliminar entradas del registro.

Escalabilidad y disponibilidad

ASP.NET controla automáticamente las fallas de código de usuario y del sistema. También se recupera automáticamente de violaciones de acceso, faltas de memoria, y deadlocks.

ASP.NET le permite configurar el número máximo de solicitudes que pueden ser puestas en cola para una aplicación antes de que se lance un nuevo proceso. En este evento las solicitudes se reasignan automáticamente al nuevo proceso. De igual forma, puede configurar un límite de memoria para el proceso de la aplicación. Si se excede este límite se lanza un nuevo proceso, y nuevamente se reasignan las solicitudes automáticamente. Eventualmente, en ambos casos, el proceso anterior se termina cuando ha terminado el procesamiento de las solicitudes pendientes.

Además, ASP.NET soporta el concepto de reinicio prioritario de aplicaciones. Este elimina la necesidad de reiniciar el servidor una vez por semana solo para asegurarse de que la aplicación sigue ejecutándose. ASP.NET puede reiniciar el proceso ASP.NET con base en el número de solicitudes servidas o la duración que ha sido ejecutada la aplicación.

Modelo de Aplicaciones de Formularios Windows

Los desarrolladores que escriben aplicaciones Windows clientes pueden utilizar el modelo de aplicaciones de formularios Windows para aprovechar las ricas características de la interfaz de usuario proporcionadas por Windows, incluyendo los controles COM existentes y las nuevas características de Windows 2000, tales como ventanas transparentes, con capas y flotantes. Puede seleccionar una apariencia Windows o Web tradicional. Los desarrolladores encontrarán muy intuitivo el modelo de programación Windows Forms y su soporte en tiempo de diseño, dadas las similitudes con los paquetes existentes de formularios basados en Windows.

Windows Forms forma parte de la plataforma .NET y utiliza muchas de las nuevas tecnologías incluyendo un entorno de aplicación común, ambiente de ejecución administrado, seguridad integrada y principios de diseño orientado a objetos.

Además, Windows Forms le permite consumir servicios Web y desarrollar ricas aplicaciones basadas en datos con soporte del modelo de datos ADO.NET. Con el nuevo ambiente de desarrollo compartido en Visual Studio.NET, puede crear aplicaciones Windows Forms utilizando cualquiera de los lenguajes que soportan la plataforma .NET.

Herencia Visual

La herencia visual es una de las características clave disponibles en Windows Forms que mejoran la productividad del desarrollador y le permiten el reuso de código. Por ejemplo, una organización puede definir un formulario base estándar que contiene elementos tales como el logotipo corporativo y quizás la barra de herramientas común. Este formulario puede ser utilizado por los desarrolladores a través de herencia y ampliado a fin de cumplir los requerimientos de las aplicaciones específicas, al tiempo que promueve una interfaz de usuario común a través de toda la organización. El creador del formulario base puede especificar qué elementos pueden ser ampliados y qué elementos deben ser utilizados sin cambios, asegurándose que el formulario se reutilice adecuadamente. Esta funcionalidad es bien conocida por los desarrolladores de Visual FoxPro.

Diseño de Formularios de Precisión

Los desarrolladores cuentan con un nivel sin precedentes de control y productividad cuando diseñan la apariencia de sus aplicaciones Windows Forms. Las características tales como In-Place Menu Editor, Control Anchoring, Control Docking, y muchos nuevos controles permiten un mayor nivel de poder y precisión para que los desarrolladores creen ricas interfaces de usuario basadas en Windows.

Con el In-Place Menu Editor, los desarrolladores pueden agregar rápida y fácilmente los menús al formulario, modificarlos y ver como se ven sin tener que ejecutar la aplicación. Los controles en el formulario son más efectivos con Control Anchoring, permitiendo a un formulario redimensionar automáticamente los controles a media que el usuario redimensiona al formulario. Con Control Docking, los controles se pueden adosar a cualquier lado del formulario proporcionando

mayor flexibilidad al diseño.

Los controles COM existentes se pueden utilizar y ejecutar en cualquier formulario, conservando las inversiones en la tecnología existente.

Nuevos controles (incluyendo Link Label, Tray Icon y Print Preview) proporcionan funcionalidad adicional común para los desarrolladores. El Link Label proporciona vinculación como con HTML a un URL específico. El texto mostrado utilizando este control aparecerá subrayado y el cursor cambiará a una forma de mano a medida que se mueve el mouse, activando un evento accionable cuando se hace clic en él. Tray Icon permite que los desarrolladores creen aplicaciones que se ejecutan en la barra de Windows, similares a Microsoft SQL Server Service Manager. Windows Forms ofrece un entorno de impresión que hace simple el imprimir, incluyendo una ventana de Presentación Preliminar de impresión con el control Print Preview.

Los desarrolladores pueden crear aplicaciones con Windows Forms que soporten la más vasta audiencia de usuarios. Los controles de Windows Forms implementan las interfaces de programación de Microsoft Active Accessibility, haciendo simple y directo el desarrollar aplicaciones que soporten accesibilidad, tales como lectores de pantalla.

Bajo Costo Total de Propiedad

Windows Forms proporciona más que solo una forma de desarrollar ricas soluciones basadas en Windows. También se beneficia de las sencillas capacidades de implementación, y de un modelo de seguridad de aplicaciones integrado. Windows Forms aprovecha el versionamiento y características de implementación de la plataforma Microsoft .NET para ofrecer costos de implementación reducidos y mayor solidez de la aplicación con el tiempo. Esto disminuye significativamente los costos de mantenimiento (costo total de propiedad) para aplicaciones escritas utilizando Windows Forms.

Con una aplicación Windows Forms, no hay necesidad de implementar una aplicación en el escritorio del usuario final. En su lugar, el usuario puede ejecutar la aplicación simplemente escribiendo el URL en el explorador. La aplicación será descargada a la máquina del cliente, ejecutada en un ambiente de ejecución seguro y será eliminada por si misma al terminar.

Para organizaciones que desean implementar físicamente una aplicación en el escritorio, no hay necesidad de realizar un proceso de instalación consumidor de recursos. Los usuarios o los administradores simplemente pueden copiar la aplicación al escritorio.

Visual Studio.NET

Visual Studio.NET soporta una gran mejora en la interfaz: barras de herramientas adosables y ocultables. Estas barras se ocultan al costado de la pantalla como solapas hasta que tenga que utilizarlas, proporcionando un espacio de pantalla más utilizable.

Otra mejora es que Integrated Development Environment (IDE) ahora unifica todos los lenguajes. Algunas de las características más tratables incluyen:

- La capacidad de colapsar y ampliar procedimientos de manera que pueda ocultar el código que no necesite ver mientras trabaja en otras secciones.
- Si tiene múltiples proyectos abiertos, pueden ser escritos en diferentes lenguajes, y el IDE irá cambiando el contexto a medida que se mueva entre ellos.
- Una lista de tareas muestra todos juntos los errores de compilación, junto con notas definidas por el usuario; al hacer doble clic en una entrada de la lista "por hacer" se brinca al segmento de código correspondiente.
- Auto-indenting se amplía para indentar automáticamente las estructuras de control.

Otro de los beneficios del IDE unificado es un conjunto unificado de diseñadores visuales, incluyendo HTML, XML, datos y diseñadores de código del lado del servidor.

Visual Studio.NET incluye un único modelo de proyecto unificado. Este nuevo modelo le permite integrar componentes tanto desde VB.NET como desde Extensiones Administradas para C++ en un proyecto en el que trabaja su equipo. Como resultado, su proyecto podrá incorporar componentes contruidos en el lenguaje que tenga más sentido.

La ayuda dinámica proporciona otra mejora valiosa. El IDE sabe dónde se encuentra dentro de su entorno y qué tarea está realizando. Esto le ayuda a aprender nuevas características y a completar comandos casi en la misma forma que IntelliSense proporciona ayuda para métodos y propiedades. El sistema de ayuda no solo proporciona ayuda a nivel de diccionario (como una función IntelliSense) sino que también proporciona consejos de programación de nivel más alto sobre cómo utilizar la herramienta.

Lenguajes

Todos los lenguajes .NET emplean el .NET Common Language Runtime (CLR) y las Base Class Library (BCL), aplicando un conjunto común de características mínimas definidas por el Common Language Specification (CLS). Asimismo, cada lenguaje soporta el Common Type System (CTS) proporcionando compatibilidad a través de lenguajes. Mientras ciertos lenguajes soportan nativamente el CTS, otros tales como C++ administrado proporcionan soporte a través de extensiones para el lenguaje base.

Por ejemplo, uno de los diseños principales de Visual Basic es ocultar las tareas de administración de memoria del desarrollador para facilitar la programación. Esto es contrario al enfoque de C++. Mientras que las nuevas características de Colección de Basura (GC) del .NET Framework son soportadas naturalmente por Visual Basic.NET, C++ requiere modificación y extensiones, las cuales son proporcionadas por Managed C++.

Opción de lenguaje

Dentro de CLR, todos los lenguajes comparten un gran conjunto de recursos entre los que se incluyen:

- Un modelo de programación orientado a objetos (herencia, polimorfismo, manejo de excepciones y colección de basura)
- Modelo de seguridad
- Sistema de tipos
- Base Class Library (BCL) (Biblioteca de Clases Base)
- Desarrollo, depuramiento y herramientas de perfilamiento
- Administración de ejecución y código
- Traductores y optimizadores de MSIL a código nativo

El Common Language Specification (CLS) es el subconjunto del Common Type System (CTS) en común entre los lenguajes, codificado como un conjunto de reglas. Los lenguajes que se atienen a estas reglas pueden interoperar por completo entre sí. Por ejemplo, puede crear una clase base en C#, y derivar la nueva clase desde ahí utilizando Visual Basic.Net.

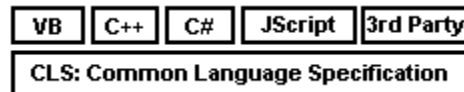


Figura. 11: CLS y Lenguajes

Puede escribir código para la plataforma .NET utilizando Microsoft Visual C++. Sin embargo, debido a que el código C++ no puede ser administrado directamente por CLR y no se alinea con el CLS, Microsoft ha agregado un conjunto de extensiones administradas a Visual C++. El código escrito con estas extensiones cumple con el CLS.

Este alto grado de interoperabilidad de lenguajes afecta fundamentalmente la manera en que elige los lenguajes de programación e implementa los diseños. La opción del lenguaje se vuelve más una preferencia personal y depende de la sintaxis que crea más adecuada. El CLR también puede canalizar una aceptación más amplia de lenguajes especiales, debido a que tendrán las mismas capacidades que los lenguajes más conocidos.

La distinción en términos de desempeño y capacidades entre Visual Basic.NET y C# es significativamente menor entre las versiones anteriores de Microsoft Visual Basic y Visual C++. Por ejemplo:

- VB.NET y C# fueron desarrollados desde cero para el .NET Framework
- VB.NET y C# tienen capacidades muy similares en términos de poder de programación. La diferencia en la funcionalidad se basa en las audiencias respectivas de desarrolladores Visual Basic y C++; por ejemplo C# soporta incrustación entre líneas de C++ no administrado.
- VB.NET y C#, no obstante, tienen diferencias considerables en términos de experiencia de usuario. Por ejemplo, la sensibilidad a mayúsculas/minúsculas es una característica fundamental de C++ que se refleja en C#. Sin embargo,

este aspecto del lenguaje resulta en una amplia diferencia en la experiencia de usuario para el desarrollador Visual Basic que trata de aprender C# o C++.

Visual Basic.NET

Con base en el sistema de tipos de .NET, Visual Basic.NET ofrece un número de mejoras importantes sobre versiones anteriores de Visual Basic. Las principales se relacionan con el soporte para las nociones de orientación a objetos como polimorfismo, herencia, y sobrecarga de operadores y métodos. Otras provienen de sus bases comunes con otros lenguajes proporcionados por CLS. Es posible que libremente pueda pasar tipos de datos a y desde componentes desarrollados en otros lenguajes, y heredar desde clases base desarrolladas en otros lenguajes. IntelliSense también trabaja con cualquier componente sin importar el lenguaje de desarrollo. Visual Basic.NET soporta ADO.NET para el acceso a datos, el modelo de programación Windows Forms, múltiples hilos de ejecución, y nuevos y mejorados asistentes y diseñadores.

Los actuales desarrolladores en Visual Basic se beneficiarán de aprender las nuevas construcciones y características del lenguaje, permitiéndoles desarrollar aplicaciones más avanzadas y robustas. Sin embargo, no tiene necesidad de actualizar todo su código, ya que el código Visual Basic 6 y el código Visual Basic.NET se ejecutará de lado a lado sin problemas. Si elige actualizar todo, los servicios de interoperabilidad COM ayudarán en gran medida con la transición por fases, permitiéndole utilizar sin problemas sus componentes existentes. Asimismo, Visual Basic.NET proporciona un asistente de actualización, el cual lo guía a través de los pasos del proceso de actualización y realiza los cambios de sintaxis de lenguaje necesarios cuando un proyecto Visual Basic 6 es cargado. Para más información de actualización de aplicaciones Visual Basic 6, por favor visite: <http://msdn.microsoft.com/library/techart/vb6tovbdotnet.htm>.

Las principales nuevas características presentadas por Visual Basic.NET incluyen:

- Herencia. A través de la herencia de implementación, las clases pueden ampliar (o heredar) desde una clase base existente. Visual Basic.NET (junto con todos los lenguajes .NET) únicamente soporta herencia simple, dando entender con esto que se puede heredar únicamente desde una sola clase base. Las nuevas clases pueden obtener la funcionalidad y comportamiento de la clase base, y preferir adaptar el comportamiento cuando corresponda. Asimismo, una clase derivada puede elegir proporcionar nuevos comportamientos, por ejemplo mediante la implementación de nuevos métodos. Para utilizar herencia de implementación, se declara la nueva clase y se utiliza la palabra reservada *Inherits* para expresar de qué clase base se deriva. Posteriormente utiliza la palabra reservada *Overrides* para modificar la funcionalidad de la clase base cuando corresponda. Observe que un método es capaz de ser sobrescrito en una clase derivada, debe ser marcado como *Overridable* en la clase base. Ésto se muestra en el siguiente código de ejemplo:

```

Public Class class1
    Overridable Function Name() As String
        ' Implementation goes here
    End Function
End Class

Public Class class2
    Inherits class1
    Overrides Function Name() As String
        ' Overriden Implementation goes here
    End Function
End Class

```

- Polimorfismo. Esta es la capacidad de tratar a los objetos de diferentes tipos de clase de la misma forma. Tradicionalmente, podría lograr esto en Visual Basic utilizando interfaces. Al tiempo que Visual Basic.NET sigue soportando el polimorfismo a través de interfaces, ahora también se puede lograr utilizando herencia.
- Constructores. Los constructores permiten que se cree un objeto y se inicie a un estado en una sola operación. Esto alivia la necesidad de crear un objeto, después llamar un método separado para inicializarlo. En Visual Basic.NET, los constructores de objetos le permiten pasar valores para un objeto mientras está siendo creado. Esto simplifica la codificación y ayuda a reducir errores cuando se trabaja con objetos.
objPerson = New Person ("Chris", "Smith", 1966) ' create and initialize new person object
- Inicializadores. Los inicializadores le permiten declarar una variable y proporcionarle un valor inicial en una sola línea de código. Esto permite lograr código más pequeño, más simple, y más mantenible:
Private mProp As Int32 = 72
Dim sTitle As String = "Marketing"
Dim dblRates() As Double = {5.3, 55.8, 82.2}
- Manejo estructurado de errores. El CLR de .NET utiliza excepciones para manejar las condiciones de error de manera consistente, sin importar el lenguaje de programación que utilice. Las excepciones son respuestas a las condiciones de falla que pueden ocurrir mientras una aplicación se esté ejecutando. Pueden ser generadas por el Runtime de .NET (excepciones del sistema) o creadas programáticamente (excepciones de la aplicación). Las excepciones se pueden propagar más allá de los límites de los lenguajes. Por ejemplo, un componente escrito en C# podría generar una excepción con la que podría ser manejado subsecuentemente por un programa cliente desarrollado con Visual Basic.NET. Este código del sitio Web de Microsoft muestra la sintaxis Visual Basic.NET para usar el control estructurado de excepciones:

```

Sub SEH()
Try
    Open "TESTFILE" For Output As #1
    Write #1, CustomerInformation
Catch
    Kill "TESTFILE"
Finally
    Close #1
End Try
End Sub

```

La construcción Try...Catch...Finally hace mucho más sencilla la localización de manejo de errores alrededor de código que pudiese causar daño.

C# (C Sharp)

Durante las últimas dos décadas, C y C++ han sido los lenguajes más ampliamente utilizados para el desarrollo de software comercial de negocios. Mientras que ambos lenguajes proporcionan al programador una vasta cantidad de control al detalle, esta flexibilidad se refleja en costo de productividad. En comparación con un lenguaje como Visual Basic, las aplicaciones C y C++ equivalentes con frecuencia tardan más en desarrollarse. Debido a la complejidad y largos ciclos de tiempo relacionados con estos lenguajes, muchos programadores C y C++ han estado buscando un lenguaje que ofrezca mejor balance entre poder y productividad.

Hoy en día, existen lenguajes que elevan la productividad sacrificando la flexibilidad que los programadores C y C++ con frecuencia requieren. Dichas soluciones restringen demasiado al desarrollador, por ejemplo, al omitir mecanismos que permiten el control de código de bajo nivel y proporcionan capacidades del menor común denominador. No interoperan fácilmente con los sistemas existentes, y no siempre se combina bien con las prácticas de programación Web actuales.

La solución ideal para los programadores C y C++ es el desarrollo rápido, combinado con el poder de acceder a toda la funcionalidad de la plataforma base. Ellos desean un ambiente que esté completamente sincronizado con los estándares Web, y que proporcione fácil integración con aplicaciones existentes. Adicionalmente, los desarrolladores C y C++ podrían desear tener la capacidad de crear código de bajo nivel si así lo desean.

C# es la solución Microsoft para este problema. C# es un moderno lenguaje orientado a objetos que permite que los programadores desarrollen rápidamente una amplia gama de aplicaciones para la nueva plataforma Microsoft .NET. Debido a su elegante diseño orientado a objetos, C# es una fabulosa opción para crear una amplia gama de componentes, desde objetos de negocios de alto nivel hasta aplicaciones de nivel del sistema. Debido a que están soportados por la plataforma .NET, estos componentes se pueden convertir en servicios Web, permitiéndoles ser invocados a través de Internet, desde cualquier lenguaje que se ejecute en cualquier sistema operativo.

C# está diseñado para proporcionar desarrollo rápido al programador C++ sin sacrificar la energía y el control que han sido un distintivo de C y C++. Debido a su herencia, C# tiene un alto grado de fidelidad con C y C++. Los desarrolladores

familiarizados con estos lenguajes pueden ser productivos más rápidamente en C#.

- Productividad y seguridad. Debido a que la nueva economía está forzando como nunca antes a los negocios a responder a amenazas competitivas más rápido, los desarrolladores son llamados a acortar los tiempos de ciclo y producir más revisiones incrementales de un programa, en lugar de una sola versión monumental. C# está diseñado con estas consideraciones en mente. El lenguaje está diseñado para ayudar a los desarrolladores a lograr más con menos líneas de código y menos oportunidades de error. Una gran característica de C# es que está diseñado para eliminar costosos errores de programación. Incluso los programadores C++ expertos pueden cometer el más simple de los errores, por ejemplo olvidando iniciar una variable, u olvidando liberar un recurso, y con frecuencia aquellos errores simples resultan en problemas no predecibles que pueden permanecer sin ser descubiertos durante largos periodos de tiempo. Una vez que el programa está en producción, puede ser muy costoso arreglar hasta el más simple error de programación. El moderno diseño de C# elimina los errores de programación C++ más comunes. Por ejemplo:

- La colección de basura libera al programador de tener que administrar memoria manualmente.
- Las variables en C# son inicializadas automáticamente por el ambiente.
- Están prohibidos los *casts* inseguros.

El resultado final es un lenguaje que hace mucho más sencillo a los desarrolladores el escribir y mantener programas que resuelven problemas de negocios complejos. También está diseñado para reducir los costos de desarrollo continuos mediante el soporte integrado para versionamiento. La actualización de componentes de software es una tarea que lleva a errores. Las revisiones hechas al código pueden cambiar no intencionalmente la semántica de un programa existente. Para asistir al desarrollador con este problema, C# incluye soporte de versionamiento en el lenguaje. Una característica relacionada es el soporte de interfaces y herencia de interfaz. Estas características permiten desarrollar entornos complejos y evolucionarlos posteriormente. Estas características hacen que el proceso de desarrollo de futuras versiones de un proyecto sean más sólidas y que con ello se reduzcan los costos globales de desarrollo para versiones sucesivas.

- Poder, expresividad y flexibilidad. C# proporciona mejor correlación entre los procesos de negocios y la implementación. Con el alto nivel de esfuerzo que las corporaciones gastan en la planeación de los negocios, es imperativo tener una conexión cercana entre el proceso de negocios abstracto y la actual implementación en software. Por ejemplo, la mayoría de las herramientas de lenguaje no tienen una forma fácil de vincular información de diseño con el código. Con frecuencia los desarrolladores utilizan comentarios en el código para identificar qué clases conforman un objeto de negocios abstracto particular.

.NET permite a un arquitecto de proyecto definir atributos específicos del

dominio y aplicarlos a cualquier elemento de lenguaje, por ejemplo clases, interfaces y demás. El desarrollador puede luego examinar programáticamente los atributos en cada elemento. Esto hace fácil, por ejemplo, el escribir una herramienta automatizada que asegurará que cada clase o interfaz esté correctamente identificada como parte de un objeto de negocios abstracto particular, o simplemente crear reportes basados en atributos específicos del dominio de un objeto. Este firme acoplamiento entre metadata personalizada y el código del programa ayuda a reforzar la conexión entre el comportamiento que se pretende con el programa y la actual implementación.

C# también proporciona extensiva interoperabilidad. El ambiente administrado y seguro en tipos es adecuado para la mayoría de las aplicaciones, pero las experiencias del mundo real muestran que las aplicaciones siguen requiriendo código "nativo", ya sea por razones de desempeño o para interoperar con las interfaces de programación de la aplicación existentes (APIs). Dichos escenarios pueden forzar a los desarrolladores a utilizar C++ incluso si prefieren utilizar un ambiente de desarrollo más productivo.

- Interoperabilidad. Ya que C# se basa en la plataforma .NET, cada objeto C# se puede utilizar como si fuera un objeto COM. Los desarrolladores ya no tendrán que implementar explícitamente interfaces IUnknown y otras de COM cuando utilizan los componentes COM en otros lenguajes. En lugar de ello, esas características están integradas. De igual forma, los programas C# pueden utilizar objetos COM existentes, sin importar el lenguaje utilizado para crearlos.
- Acceso a la plataforma. Para aquellos desarrolladores que lo requieren, C# incluye una característica especial que permite a un programa invocar cualquier API nativo de C. Dentro de un bloque de código especialmente marcado, los desarrolladores pueden utilizar punteros y características C/C++ tradicionales tales como administración manual de memoria y aritmética de punteros. Esta es una gran ventaja sobre los demás ambientes. Significa que los programadores C# pueden desarrollar sobre su base de código C y C++ utilizando las Extensiones Administradas para C++, en lugar de tener que descartar el código existente.
- Estandarización de C#. Cuando Microsoft anuncia la referencia del lenguaje C#, simultáneamente anunció la propuesta de estandarización del lenguaje. C# se somete formalmente como una propuesta para el ECMA Technical Committee (TC) 39. Este es el mismo comité que estandariza ECMAScript (JScript).

Asumiendo que los miembros del comité acepten la presentación y estén de acuerdo con estandarizar C#, otros proveedores además de Microsoft podrán implementar el lenguaje libremente. Sin duda, se verán proveedores implementando "C# estándar" y sus extensiones C# propias, tal como hoy en día implementan C y C++ estándar junto con sus extensiones propietarias. En este sentido, se espera que el eventual estándar C# imitará el balance de los estándares C y C++ entre portabilidad y la invención del proveedor.

-
- Un real lenguaje de desarrollo basado en componente. C# simplifica el desarrollo basado en componentes. Los conceptos basados en componentes tales como propiedades, eventos, versionamiento y *meta-data*, son ciudadanos de primera clase integrados en el lenguaje. Como resultado, no hay necesidad de un encabezado externo y de archivos IDL junto con los GUIDs asociados y definiciones de interfaz. Tampoco hay necesidad de métodos nombrados específicamente, utilizados para crear la ilusión de una propiedad. Un componente deberá ser autodescriptivo. Esto se facilita principalmente mediante *meta-data*, el cual no se puede separar del componente (ya que es parte del ensamble del componente). El programador también tiene la opción de incorporar etiquetas de comentarios XML en el código de fuente. Estas etiquetas de código fuente pueden ser utilizadas por el compilador para generar documentación basada en XML.

Extensiones administradas para C++

Las extensiones administradas para C++ son un conjunto de extensiones de lenguaje que ayudan a los desarrolladores de Microsoft Visual C++® a escribir aplicaciones para la plataforma Microsoft .NET.

Las extensiones administradas son útiles si:

- Desea migrar en etapas una gran porción de código, desde C++ no administrado hacia la plataforma .NET
- Tener componentes C++ no administrados que desee utilizar desde las aplicaciones .NET Framework
- Tener componentes .NET Framework que desee utilizar desde C++ no administrado
- Desea mezclar código C++ no administrado y código .NET en la misma aplicación

Las extensiones administradas para C++ ofrecen inigualable flexibilidad para los desarrolladores que se enfocan en la plataforma .NET. El código C++ tradicional no administrado y C++ administrado se pueden mezclar libremente dentro de la misma aplicación. Las nuevas aplicaciones escritas con extensiones administradas pueden aprovechar lo mejor de ambos mundos. Los componentes existentes pueden empaquetarse fácilmente como componentes .NET utilizando las extensiones administradas, conservando la inversión en el código existente al tiempo que se integra con .NET.

¿Qué son las extensiones administradas?

Las extensiones le permiten escribir clases administradas en C++, las cuales se ejecutan bajo el control del .NET Framework. (Las clases C++ tradicionales no administradas se ejecutan en el ambiente basado en Microsoft Windows®). Una clase administrada es una clase .NET nativa y puede explotar por completo el .NET Framework.

Las extensiones administradas son nuevas palabras reservadas y atributos en el sistema de desarrollo Visual C++. Le permiten decidir que clases y funciones

compilar como código administrado o no administrado. Posteriormente, estas piezas pueden interoperar suavemente entre si y con bibliotecas externas.

Asimismo, las extensiones administradas se utilizan para expresar tipos y conceptos .NET directamente en el código fuente C++. Esto permite a los desarrolladores escribir aplicaciones .NET fácilmente sin escribir código extra.

Escenarios clave

- **Suave migración de código existente a .NET**
Si tiene una gran inversión en código C++, las extensiones administradas le ayudarán a hacer una transición suave a la plataforma .NET. Debido a que puede mezclar código no administrado y administrado en la misma aplicación (incluso en el mismo archivo) puede mover código en el tiempo, componente por componente, a .NET. O bien, puede seguir escribiendo componentes en C++ no administrado, aprovechando todo el poder y flexibilidad del lenguaje, y utilizar únicamente extensiones administradas para escribir finos empaquetadores de alto desempeño que hagan que el código C++ pueda llamarse desde los componentes .NET.
- **Accediendo un componente C++ desde un lenguaje .NET**
Las extensiones administradas permiten llamar una clase C++ desde cualquier lenguaje .NET. Necesita escribir una simple clase de empaquetamiento utilizando las extensiones que expongan su clase y métodos C++ como una clase administrada. El paquete es una clase administrada y puede ser llamada desde cualquier lenguaje .NET. La clase paquete actúa como una capa de enlace entre la clase administrada y la clase C++ no administrada. Simplemente pasa las llamadas a métodos desde código .NET administrado directamente dentro de la clase no administrada. Las extensiones administradas se pueden utilizar para llamar a cualquier biblioteca de vínculo dinámico (DLL), así como clases nativas.
- **Accediendo a clases .NET desde código nativo**
Utilizando las extensiones administradas, puede crear y llamar una clase .NET directamente desde código C++. Puede escribir código C++ que trate al componente .NET como cualquier otra clase C++ administrada. También puede utilizar el soporte COM nativo en el .NET Framework para llamar a las clases .NET. El uso de COM o las extensiones administradas para acceder a los componentes .NET, dependerá de su proyecto. En algunos casos, el utilizar el soporte COM existente será la mejor opción. En otros casos, será posible aumentar el desempeño y la productividad del desarrollador utilizando las extensiones administradas.
- **Código administrado y nativo en un mismo ejecutable**
El compilador Visual C++ traduce los datos, punteros, excepciones y flujo de instrucciones entre los contextos administrados y no administrados automáticamente y de manera transparente. Este proceso es lo que le permite a las extensiones administradas interoperar sin problemas con código C++ no administrado. El desarrollador recibe un control fino sobre qué datos y código deben ser administrados.

La posibilidad de elegir si cada clase y función va a ser administrada o no administrada brinda mayor flexibilidad. Algunos tipos de código o datos se desempeñarán mejor en un ambiente no administrado. Por otra parte, el código administrado por lo regular ofrece mayor productividad al desarrollador debido a las características tales como colección de basura y bibliotecas de clase. El código C++ no administrado existente se puede convertir en código administrado de a una sección a la vez, conservando su inversión existente.

Mejoras en Acceso a Datos

ADO.NET es una mejora evolutiva sobre Microsoft ActiveX Data Objects (ADO) que proporciona interoperabilidad entre plataformas y acceso escalable a datos. Debido a que el Extensible Markup Language (XML) es el formato para transmitir datos en ADO.NET, cualquier aplicación que pueda entender XML puede procesar datos. La aplicación que los recibe puede estar ejecutándose en cualquier plataforma.

Con Visual Studio.NET, puede programar contra sus objetos, y no contra tablas y columnas. ADO.NET utiliza programación de tipos forzada en la cuál los objetos de negocios figuran de manera prominente.

Por ejemplo, considere la siguiente línea de código, utilizando programación convencional (no de tipos forzados):

```
IF TotalCost > Table("Customer").Column("AvailableCredit")
```

En este ejemplo, está programando las tablas y columnas ADO. Con la programación forzada de tipos, el mismo ejemplo es mucho más sencillo de leer y escribir:

```
IF TotalCost > Customer.AvailableCredit
```

DataSets de ADO.NET

La pieza central de cualquier solución de software utilizando ADO.NET, es el DataSet. Un DataSet es una copia en memoria de datos de base de datos. Un DataSet contiene cualquier número de tablas de datos, donde cada una corresponde típicamente a una tabla de base de datos o vista, junto con cualquiera de las relaciones y restricciones asociadas. Un DataSet constituye una vista de los datos desconectada de la base de datos. Existe en la memoria sin una conexión activa a una base de datos que contenga las tablas o vistas correspondientes.

En tiempo de ejecución, los datos se pasarán desde una base de datos hasta un objeto de negocios de capa intermedia y después a la interfaz de usuario. Para recibir el intercambio de datos, ADO.NET utiliza un formato de persistencia y transmisión basado en XML. Para transmitir los datos desde una capa a otra, la solución ADO.NET expresa los datos en la memoria (el conjunto de datos) como XML y después se envía XML al otro componente.

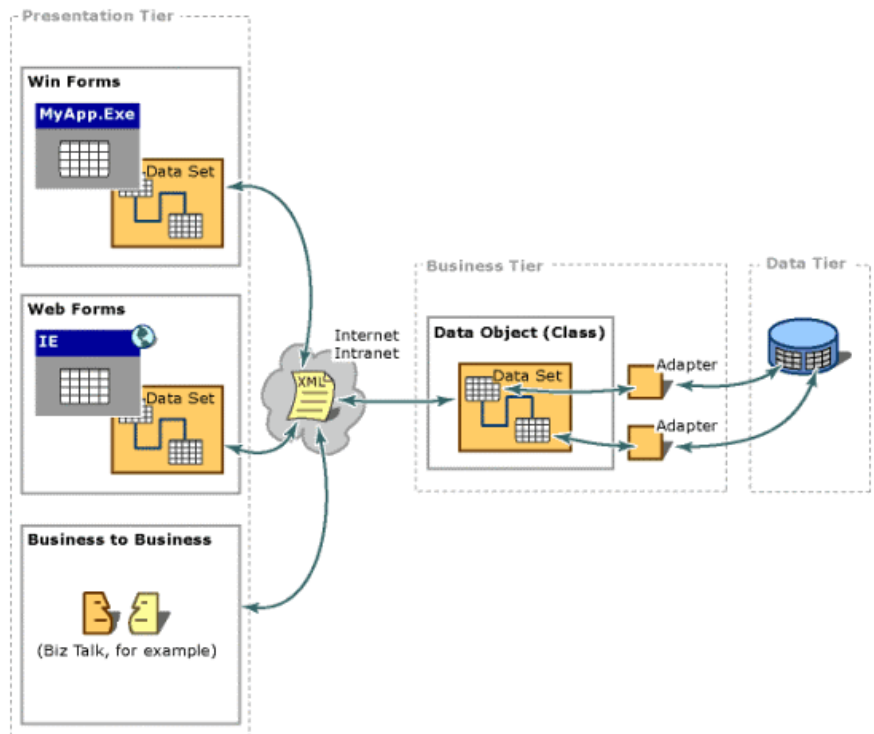


Figura. 12: Principales componentes de una solución ADO.NET.

Para trabajar fácilmente con datos en Visual Studio.NET, existen varias nuevas características. Para el desarrollador XML experimentado, se proporciona un XML Designer con código rico en colores, con terminación automática de frases y etiquetas.

Para una vista gráfica de los datos, los desarrolladores pueden utilizar la vista de diseño del XML Designer. Esto le permite arrastrar y colocar tablas desde cualquier fuente de datos, incluyendo Microsoft SQL Server y bases de datos Oracle, desde el Server Explorer hasta la superficie de diseño. Puede crear DataSets que están conformados de datos desde diversas fuentes, incluyendo archivos XML.

Con frecuencia necesita agregar, modificar o eliminar datos mientras diseña su aplicación. Con la opción Data Preview, no solo puede agregar y modificar datos, sino que también puede navegar las relaciones de sus datos.

Las tecnologías de vinculación a datos para Visual Studio.NET han sido drásticamente mejoradas para aprovechar al máximo a ADO.NET. Por lo que es fácil crear las interfaces de usuario que interactúan con datos. Lo que es más importante, ahora puede vincular valores con objetos de negocios y servicios Web.

Compartir Datos con ADO.NET

Para acomodar el intercambio de datos entre los DataSet y sus respectivas fuentes de datos, una solución ADO.NET utiliza un objeto denominado comando de conjunto de datos. Microsoft proporciona dos objetos de comando de datos:

-
- El objeto `SQLDataSetCommand`: este objeto media la comunicación entre la tabla dentro de un `DataSet` y una tabla o vista en una base de datos SQL Server.
 - El objeto `ADODatasetCommand`: este objeto media la comunicación entre la tabla dentro de un `DataSet` y una tabla o vista en cualquier fuente de datos que tenga un proveedor OLE DB.

ADO.NET ofrece diversas ventajas sobre versiones anteriores de ADO y sobre otros componentes de acceso de datos. Estos beneficios se dividen en las siguientes categorías:

- **Interoperabilidad:** las soluciones ADO.NET pueden aprovechar la flexibilidad y amplia aceptación de XML. Debido a que XML es el formato para transmitir conjuntos de datos entre componentes, cualquier componente (en cualquier plataforma) que pueda leer formato XML puede procesar los datos. Cuando se transmite un registro desconectado ADO desde un componente a otro, el componente que recibe debe acomodar el ajuste de parámetros de COM. Esto es, debe ser un componente COM. Esta restricción no se aplica cuando se transmite el conjunto de datos ADO.NET desde un componente a otro. Debido a que la transmisión del conjunto de datos ocurre a través de archivos XML, y debido a que XML es un estándar simple basado en texto de amplia aceptación en la industria, el componente que recibe no tiene restricciones arquitecturales. Con esto, virtualmente cualquiera dos componentes pueden compartir conjuntos de datos, dado que ambos utilicen el mismo esquema XML para el formato basado en XML del conjunto de datos.
- **Mantenimiento:** en el tiempo de vida de un sistema implementado, son posibles algunos pequeños cambios. Pero cambios sustanciales de arquitectura son raramente intentados debido a que son muy difíciles. Esto es desafortunado, debido a que en el curso natural de eventos, dichos cambios sustanciales pueden ser necesarios. Por ejemplo, a medida que una aplicación implementada se vuelve popular dentro de la comunidad de usuarios, el incrementar el desempeño puede requerir cambios arquitecturales. A medida que crece la carga en un servidor de aplicaciones implementado, los recursos del sistema pueden volverse críticos y el tiempo de respuesta o la tasa de envío de información pueden sufrir. Enfrentados con este problema, los arquitectos de software pueden optar por dividir el procesamiento de la lógica de negocios y el procesamiento de interfaz de usuario en capas separadas en máquinas separadas. En efecto, la capa de servidor de aplicaciones se reemplaza con dos capas, aliviando la escasez de recursos del sistema.

Si la solución original se implementa en ADO.NET utilizando conjuntos de datos, esta transformación es mucho más sencilla. Recuerde, cuando reemplace un solo nivel con dos niveles, usted dispone estos dos niveles para intercambiar la información. Debido a que estos niveles pueden transmitir datos a través de conjuntos de datos formateados por XML, la comunicación es relativamente sencilla.

-
- Desempeño: para aplicaciones desconectadas, los conjuntos de datos ADO.NET ofrecen ventajas de desempeño sobre conjuntos de registros desconectados de ADO. Cuando se usa el ajuste de parámetros (marshaling) de COM para transmitir un conjunto de datos desconectados entre niveles, puede resultar muy costoso en procesamiento el convertir los valores en el conjunto de registros a tipos de datos reconocidos por COM. En ADO.NET, dicha conversión de tipo de datos no es necesaria.
 - Escalabilidad: debido a que el Web puede ampliamente aumentar las demandas a sus datos, la escalabilidad se ha convertido en un factor importante para caracterizar la calidad de sus aplicaciones. Si su aplicación utiliza una interfaz basada en Web, existe un suministro virtualmente ilimitado de potenciales usuarios. Aunque una aplicación puede servir bien a una docena de usuarios, quizá no pueda servir de igual manera a cientos de ellos. El mayor impedimento de la escalabilidad es la contención por recursos limitados. Si una aplicación consume recursos tales como bloqueos de base de datos, y conexiones a base de datos, la aplicación no servirá bien a altos números de usuarios, debido a que, a medida que el número de usuarios crece, la demanda para dichos recursos excederá eventualmente su suministro. De esta forma, una aplicación que sirve bien a docenas de usuarios, puede dar servicio a cientos de ellos pero deficientemente. ADO.NET proporciona escalabilidad orientando a los programadores a conservar los recursos limitados por los cuales luchan los usuarios. Debido a que cualquier aplicación ADO.NET emplea acceso desconectado a datos de la base de datos, no mantiene bloqueos en la base de datos o conexiones de base de datos activos por largo tiempo. Con esto, la contención de los recursos de base de datos es limitada.

Transmitir un conjunto de datos ADO.NET entre capas y componentes es más fácil que transmitir un registro desconectado ADO entre ellos. Para transmitir un conjunto de registro desconectado de ADO desde un componente a otro, se utiliza el ajuste de parámetros de COM (*marshalling*). Para transmitir un conjunto de datos ADO.NET simplemente se transmite un archivo XML. La transmisión de archivos XML ofrece las siguientes ventajas por sobre el marshaling de COM:

- Tipos de datos más ricos: el *marshalling* de COM proporciona un conjunto limitado de tipo de datos – los tipos de datos definidos por el estándar COM. Debido a que la transmisión de los conjuntos de datos se basa en el formato XML, no hay restricción en los tipos de datos. De esta forma, los componentes que comparten el conjunto de datos pueden utilizar cualquier conjunto de datos rico en tipos de datos.
- Desempeño: transmitir un conjunto de registros ADO grande o un conjunto de datos ADO.NET grande puede consumir recursos de red. A medida que aumenta la cantidad de datos, la tensión que se coloca en la red también aumenta. Tanto ADO como ADO.NET le permiten minimizar los datos que son transmitidos.

No obstante, ADO.NET ofrece otra ventaja en el desempeño, y es que

ADO.NET no requiere conversiones de tipos de datos. Por el contrario, ADO, que requiere COM *marshalling* para transmitir conjuntos de registros entre componentes, requiere que los tipos de datos ADO sean convertidos a tipos de datos COM.

- Penetrar Firewalls: un *firewall* puede interferir con dos componentes tratando de transmitir los conjuntos de registros ADO desconectados. Los *Firewalls* por lo regular se configuran para permitir que el texto HTML pase, pero evita que pasen las solicitudes a nivel del sistema (tales como COM *marshalling*). Ya que los componentes intercambian conjuntos de datos ADO.NET utilizando XML, los *firewalls* pueden permitir el paso de conjuntos de datos.

Acceso Desconectado a Bases de Datos

Un conjunto de datos ADO.NET proporciona acceso desconectado a los datos en la base de datos. En ADO, el conjunto de registros puede proporcionar acceso desconectado, pero por lo regular se utiliza para proporcionar acceso conectado.

Hay una diferencia importante entre el procesamiento desconectado en ADO y ADO.NET. En ADO, puede comunicarse con la base de datos haciendo llamadas a un proveedor OLE DB. Sin embargo, en ADO.NET puede comunicarse con la base de datos a través de un objeto `DataSetCommand`, el cual llama a un proveedor OLE DB (o en algunos casos, directamente a los APIs proporcionados por el Sistema de Administración de Base de Datos (DBMS)).

La diferencia importante es que puede modificar el código de un objeto `DataSetCommand`. Como resultado, puede controlar cómo los cambios en el conjunto de datos son transmitidos a la base de datos. Esto significa que puede optimizar para desempeño, realizar verificaciones de validación de datos, o agregar cualquier otro procesamiento extra.

APENDICE

Vínculos a recursos

La siguiente no es una lista completa y exhaustiva de artículos, pero le proporciona punteros a los artículos más claros acerca de .NET.

Tecnología

XML:

<http://msdn.microsoft.com/xml/default.asp>

Información COM:

http://www.microsoft.com/net/developer/framework_com.asp

Evolución Windows DNA:

<http://microsoft.com/business/products/webplatform/default.asp>

Para información acerca de UDDI:

<http://www.uddi.org>

<http://www.microsoft.com/presspass/features/2000/sept00/09-06uddi.asp>

Web Services y la plataforma .NET:

<http://msdn.microsoft.com/msdnmag/issues/0900/Framework/print.asp>

<http://msdn.microsoft.com/msdnmag/issues/1000/Framework2/print.asp>

<http://msdn.microsoft.com/msdnmag/issues/0900/WebPlatform/WebPlatform.asp>

<http://www.microsoft.com/net/xmlservices.asp>

<http://msdn.microsoft.com/msdnmag/issues/0800/webservice/print.asp>

<http://msdn.microsoft.com/msdnmag/issues/0300/soap/print.asp>

Garbage Collection

<http://msdn.microsoft.com/msdnmag/issues/1100/gci/print.asp>

Implementación y soporte de usuarios

<http://msdn.microsoft.com/msdnmag/issues/1000/metadata/print.asp>

Desarrolladores

Para tecnología y recursos de desarrolladores:

<http://msdn.microsoft.com/resources/>

Tecnología y arquitectura de .NET:

<http://www.microsoft.com/net/>

<http://msdn.microsoft.com/net/>

MSDN Magazine

<http://msdn.microsoft.com/msdnmag>

Web Services:

<http://msdn.microsoft.com/msdnmag/issues/0900/VSNET/print.asp>

<http://msdn.microsoft.com/msdnmag/issues/0900/Webplatform/print.asp>

<http://msdn.microsoft.com/msdnmag/issues/0800/Webservice/print.asp>

Desarrollo rápido de aplicaciones para el servidor:

<http://msdn.microsoft.com/vstudio/nextgen/technology/radserver.asp>

ASP.NET

<http://msdn.microsoft.com/msdnmag/issues/0900/aspplus/print.asp>

Lenguajes

<http://msdn.microsoft.com/library/welcome/dsmsdn/deep07202000.htm>

Sitios generales para visitar

<http://www.microsoft.com/net>

<http://msdn.microsoft.com/net>

<http://www.GotDotNet.com>

<http://www.asp.net>

<http://www.microsoft.com/servers/net/default.htm>

<http://www.ibuyspy.com>